本科毕业论文（设计）

RISC-V 指令集乱序处理器设计

Design of RISC-V Out-of-Order Processor

王靳

哈尔滨工业大学

2026 年 5 月

□毕业论文　□毕业设计　　密级：公开

本科毕业论文（设计）

RISC-V 指令集乱序处理器设计

本　　科　　生：王靳

学　　号：220111012

指　导　教　师：薛睿高级实验师

专　　业：计算机科学与技术

学　　院：深圳校区计算机科学与技术学院

答　辩　日　期：2026 年 5 月

学　　校：哈尔滨工业大学

摘 要

随着开源指令集架构 RISC-V 的快速发展，基于开源生态开展高性能处理器研究已成为体系结构领域的重要方向。乱序执行通过动态调度与寄存器重命名提升指令级并行性，是现代处理器获得高性能的关键机制。本项目围绕“RISC-V 指令集乱序处理器设计”，完成了从核心微架构到系统软件支撑的整体实现与验证。

在微架构设计方面，本项目实现了前后端解耦的单发射乱序处理器，采用统一物理寄存器的寄存器重命名方案，支持乱序执行与顺序提交，并通过重排序机制实现精确异常；前端分支预测采用 GShare+IJTC+RAS 组合方案。存储层次方面，实现了基于 PIPT 结构且采用 PLRU 替换策略的 ICache 与 DCache，以降低访存开销并兼顾地址转换场景下的访问效率。体系结构支持方面，实现了 M/S/U 三级特权架构、Sv39 虚拟内存机制与中断处理机制，为操作系统运行提供了完整硬件基础。

为保证设计正确性，本项目基于 Chisel、Verilator 与 C++20 构建验证平台，结合差分测试与调试追踪工具对关键模块进行系统验证。实验结果表明，所实现处理器在乱序执行、缓存访问、异常中断处理与虚拟内存管理等关键功能上均满足设计预期，并已成功运行 xv6 教学操作系统。本项目工作为 RISC-V 乱序处理器的教学与工程实践提供了可复现的实现路径与系统级参考。

关键词： RISC-V；乱序执行；Chisel；SoC；差分测试

Abstract

With the rapid development of the open-source RISC-V instruction set architecture, conducting high-performance processor research within an open ecosystem has become a significant direction in computer architecture. Out-of-order execution improves instruction-level parallelism through dynamic scheduling and register renaming, serving as a key mechanism for achieving high performance in modern processors. This thesis presents a complete design and verification of a RISC-V out-of-order processor, spanning from core microarchitecture to system software support.

In terms of microarchitectural design, this work implements a single-issue out-of-order processor with a decoupled frontend-backend architecture. The processor employs a unified physical register file based register renaming scheme, supports out-of-order execution with in-order commit, and achieves precise exceptions through a reorder buffer mechanism. The frontend branch prediction adopts a combined GShare+IJTC+RAS scheme. For the memory hierarchy, PIPT-structured ICache and DCache with Pseudo-LRU replacement policy are implemented to reduce memory access latency while accommodating address translation scenarios. In terms of architectural support, M/S/U three-level privilege modes, the Sv39 virtual memory mechanism, and interrupt handling are implemented, providing a complete hardware foundation for operating system execution.

To ensure design correctness, a verification platform is built using Chisel, Verilator, and C++20, incorporating differential testing against the Spike reference model along with debugging and tracing tools for systematic module-level verification. Experimental results demonstrate that the implemented processor meets design expectations across all key functionalities including out-of-order execution, cache access, exception and interrupt handling, and virtual memory management. The processor has successfully booted the xv6 teaching operating system. This work provides a reproducible implementation path and system-level reference for RISC-V out-of-order processor education and engineering practice.

Keywords: RISC-V, Out-of-Order Execution, Chisel, SoC, Difftest

目　录

摘　要 I
Abstract II
第 1 章　绪　论 1
1.1　研究背景与意义 1
1.2　乱序处理器及其相关理论的发展概况 1
1.2.1　乱序执行理论框架的形成与完善 1
1.2.2　精确异常的实现路径 1
1.2.3　寄存器重命名机制与 WAW/WAR 消解 1
1.2.4　分支预测与前端指令供应能力提升 2
1.2.5　开源 RISC-V 与 Chisel 驱动下的新发展 2
1.3　本项目的主要研究内容 2
第 2 章　SoC 系统设计思路 3
2.1　设计目标与总体原则 3
2.2　分层总线架构设计 3
2.3　多层次存储与启动链路设计 5
2.4　外设集成与可视化验证思路 5
2.5　本章小结 6
第 3 章　关键 IP 核设计与实现 7
3.1　Flash 控制器与 XIP 机制 7
3.2　PSRAM QSPI Master 控制器 7
3.3　SDRAM 控制器 8
3.4　本章小结 10
第 4 章　前后端解耦架构 11
4.1　前后端解耦架构设计 11
4.2　前后端解耦架构的含义 11
4.3　本章小结 12
第 5 章　前端：取指单元与动态分支预测设计 13
5.1　取指单元（IFU）状态机设计 13
5.2　动态分支预测单元设计 14
5.2.1　GShare 分支方向预测器（BHT） 15
5.2.2　间接跳转目标缓存（IJTC） 15
5.2.3　返回地址栈（RAS） 15
5.2.4　预测选择逻辑 15
5.2.5　与 BTB 方案的对比及设计取舍 15
5.2.6　预测器更新策略 16

5.3 本章小结 16
第 6 章 后端：乱序执行引擎 17
6.1 译码级 17
6.2 寄存器重命名 18
6.3 分派级 19
6.4 发射队列与乱序发射 20
6.5 执行单元 21
6.6 延迟执行单元 21
6.7 提交级 22
6.8 本章小结 22
第 7 章 RISC-V 特权级架构 24
7.1 M/S/U 三级特权架构 24
7.2 CSR 寄存器文件 24
7.3 精确异常 25
7.4 中断机制 25
7.4.1 中断源与挂起检测 25
7.4.2 中断使能与委托 26
7.4.3 中断注入与优先级 26
7.4.4 MRET 与 SRET 26
7.5 Sv39 虚拟内存与内存管理单元 26
7.5.1 TLB 设计 27
7.5.2 PTW 设计 27
7.5.3 IFU 中的地址翻译 28
7.5.4 LSU 中的地址翻译 28
7.6 本章小结 29
第 8 章 缓存设计 30
8.1 类 SRAM 总线协议 30
8.1.1 Diplomacy 参数协商 31
8.1.2 SRAMToAXI4 桥接与缓存的双面接口 32
8.2 PLRU 替换策略 32
8.3 ICache 33
8.4 DCache 34
8.5 指令-数据缓存一致性与 fence.i 35
8.6 本章小结 36
第 9 章 仿真平台 37
9.1 仿真平台整体架构 37
9.2 简易调试器 38
9.2.1 表达式求值引擎 38
9.3 执行追踪 39

9.3.1 ITrace：指令追踪 39
9.3.2 MTrace：访存追踪 39
9.3.3 DTrace：设备追踪 39
9.3.4 FTrace：函数调用追踪 39
9.4 差分测试 39
9.4.1 理论基础：处理器的双层状态机模型 39
9.4.2 参考模型集成 40
9.4.3 比对流程 40
9.4.4 MMIO 特殊处理 40
9.4.5 失败诊断 40
9.5 LightSSS 快照回放 41
9.6 NVBoard 外设可视化 41
9.7 本章小结 42
结 论 43
参考文献 45
攻读学士学位期间取得创新性成果 47
原创性声明和使用权限 48
致 谢 49

第 1 章　绪　论

1.1　研究背景与意义

RISC-V 作为开源、模块化、可扩展的指令集架构[1]，正在从学术研究走向产业化部署。乐鑫 ESP32-C3 等商用芯片已采用 RISC-V 内核[2]，NVIDIA 于 2024 年出货超十亿颗 RISC-V 核心[3]，国内政策层面亦明确鼓励 RISC-V 生态建设[4]。相较于传统封闭指令集，RISC-V 在授权成本、生态开放性和可定制能力方面具有显著优势，为高校和科研机构开展处理器全栈设计提供了可行路径。与此同时，面向高性能场景的处理器普遍依赖乱序执行技术来提升指令级并行性和流水线吞吐率，因此围绕 RISC-V 乱序处理器开展研究具有明确的工程价值与教学价值。

从产业需求看，掌握高性能处理器核心微架构的自主设计能力，是推进自主可控芯片技术体系建设的重要基础[5]。从教学与科研实践看，当前公开资料中存在“简单顺序核容易入门、工业级乱序核复杂度过高”的鸿沟，不利于学习者系统理解乱序执行关键机制。蜂鸟 E203[6] 等开源核聚焦于顺序执行，而 SonicBOOM[7] 与香山[8] 等高性能乱序核的复杂度远超教学场景。国内已有多部教材对顺序流水线处理器设计进行了系统介绍[9,10]，姚永斌的《超标量处理器设计》[11] 则对乱序微架构理论进行了深入讲解。基于此，本项目以可实现、可验证、可复现为导向，设计并实现一款面向教学与工程过渡场景的 RISC-V 乱序处理器与配套 SoC 平台。

1.2　乱序处理器及其相关理论的发展概况

1.2.1　乱序执行理论框架的形成与完善

20 世纪 90 年代是现代乱序处理器理论体系系统化的关键时期。Smith 与 Sohi 在 1995 年的综述[12] 中指出，高性能超标量处理器的核心任务是在保持顺序语义可见性的前提下，将程序中的潜在并行性转化为硬件可执行的并行操作。该综述将处理流程归纳为取指与分支处理、寄存器相关性分析、动态发射与执行、访存数据通信、按程序顺序提交与精确中断支持等关键阶段，为后续乱序微架构设计提供了统一分析框架。

1.2.2　精确异常的实现路径

为在乱序执行条件下保持体系结构可见状态的顺序一致性，精确异常机制是核心设计点。典型实现路径包括：基于 ROB 的顺序提交方案[13]（在提交点统一更新架构状态）；基于 walk-back 的恢复方案（异常时按记录回退未提交状态）；基于 checkpoint 的快速恢复方案[14]（保存关键映射快照并在错误路径上快速回滚）。三类方案分别在硬件开销、恢复时延与实现复杂度之间进行权衡。

1.2.3　寄存器重命名机制与 WAW/WAR 消解

为消除 WAW/WAR 等伪相关并释放指令级并行性，重命名机制形成了多条技术路线。第一类是 ROB-based 方案，结果先写入 ROB 项并在提交时回写架构寄存器；第二类是 Arch Register File + Future File 方案，将提交态与推测态分离管理；第

三类是统一物理寄存器文件（PRF）方案，通过 RAT 维护映射关系并在提交后回收旧物理寄存器。统一 PRF 方案因扩展性与实现弹性较好，在 Alpha 21264[15]、MIPS R10000[16] 等经典乱序处理器中应用广泛。

1.2.4 分支预测与前端指令供应能力提升

分支预测能力直接决定前端取指带宽的有效利用率。相关技术从静态预测与一/二位饱和计数器，发展到两级自适应预测（如 GShare、GSelect）[17]，再到混合预测与锦标赛选择机制[18]。同时，BTB、RAS 与间接跳转目标缓存等结构被联合用于覆盖条件分支、函数返回与间接跳转场景。随着乱序窗口扩大与流水线加深，分支预测准确率与误判恢复代价已成为影响处理器实际吞吐率的关键因素。

1.2.5 开源 RISC-V 与 Chisel 驱动下的新发展

近年来，RISC-V 开源生态显著降低了高性能处理器研究的实现门槛，推动了乱序处理器从“理论可讲”向“工程可复现”转变。以 BOOM[7]、香山[8] 等项目为代表的开源实践展示了乱序处理器在现代开源工具链下的系统化实现路径，也为后续研究提供了可参考的架构与验证经验。

同时，Chisel[19] 等硬件构造语言的发展，使复杂微架构与 SoC 集成的参数化设计、模块复用和工程维护能力明显提升。与之配套的差分测试方法[20]（如基于 Spike[21] 的逐指令对比）进一步提高了功能验证效率，形成了“设计—验证—迭代”闭环。上述新发展使得在课程与科研场景中实现具备完整特权架构、缓存与虚拟内存支持的乱序处理器成为现实可行的工程目标。

1.3 本项目的主要研究内容

围绕上述理论与工程发展脉络，本项目完成了面向 RISC-V 的单发射乱序处理器与 SoC 系统实现，主要成果如下：

1. 完成 SoC 系统设计与集成，建立了处理器核、总线、存储与外设协同工作的系统级平台。

2. 基于 Chisel[19] 完成关键 IP 核实现，包括 SDRAM 控制器、Flash 控制器（SPI 模式，配合 XIP 控制器实现直接取指执行）与 PSRAM 控制器（支持 QSPI 与 QPI 模式及相互切换）。

3. 完成单发射乱序处理器设计，采用统一物理寄存器重命名方案，实现乱序执行、顺序提交与精确异常支持。

4. 完成基于 PIPT 结构且采用 PLRU 替换策略的 ICache 与 DCache 设计与实现。

5. 完成 RISC-V M/S/U 三级特权架构与 Sv39 虚拟内存机制实现，具备操作系统运行所需的体系结构支持能力。

6. 完成 NVBoard（NJU Virtual Board）集成，实现外设可视化联调与系统级交互验证。

7. 完成 xv6[22] 教学操作系统移植与运行验证。

第 2 章　SoC 系统设计思路

2.1　设计目标与总体原则

本课题 SoC 设计的核心目标是为乱序处理器提供可运行操作系统的完整系统环境，并在保证可调试性的前提下兼顾性能与实现复杂度。为此，系统采用“分层总线 + 多层次存储 + 可视化外设联调”的总体思路：高速访存路径优先保障带宽与时延，低速外设路径优先保障协议兼容与工程可维护性，同时通过统一验证框架降低系统集成阶段的调试成本。

在实现策略上，SoC 结构遵循模块化和可替换原则。处理器核、总线互连、存储控制器和外设控制器通过标准接口连接，使得各模块可以独立迭代。该设计既满足当前单发射乱序处理器的系统集成需求，也为后续微架构升级和外设扩展预留了接口空间。

2.2　分层总线架构设计

SoC 采用 AXI4[23] 与 APB 组合的分层互连架构，以匹配不同设备的速率特征。处理器核心通过 AXI4 主接口接入一级 Crossbar，一级 Crossbar 重点服务主存访问关键路径，仅挂接高带宽存储控制器；二级 Crossbar 连接片上存储与桥接模块，承担中速设备访问；低速外设统一通过 AXI4ToAPB 桥进入 APB 域，由 APB Fanout 完成地址分发。

SoC 整体互连架构如 图 2-1 所示。

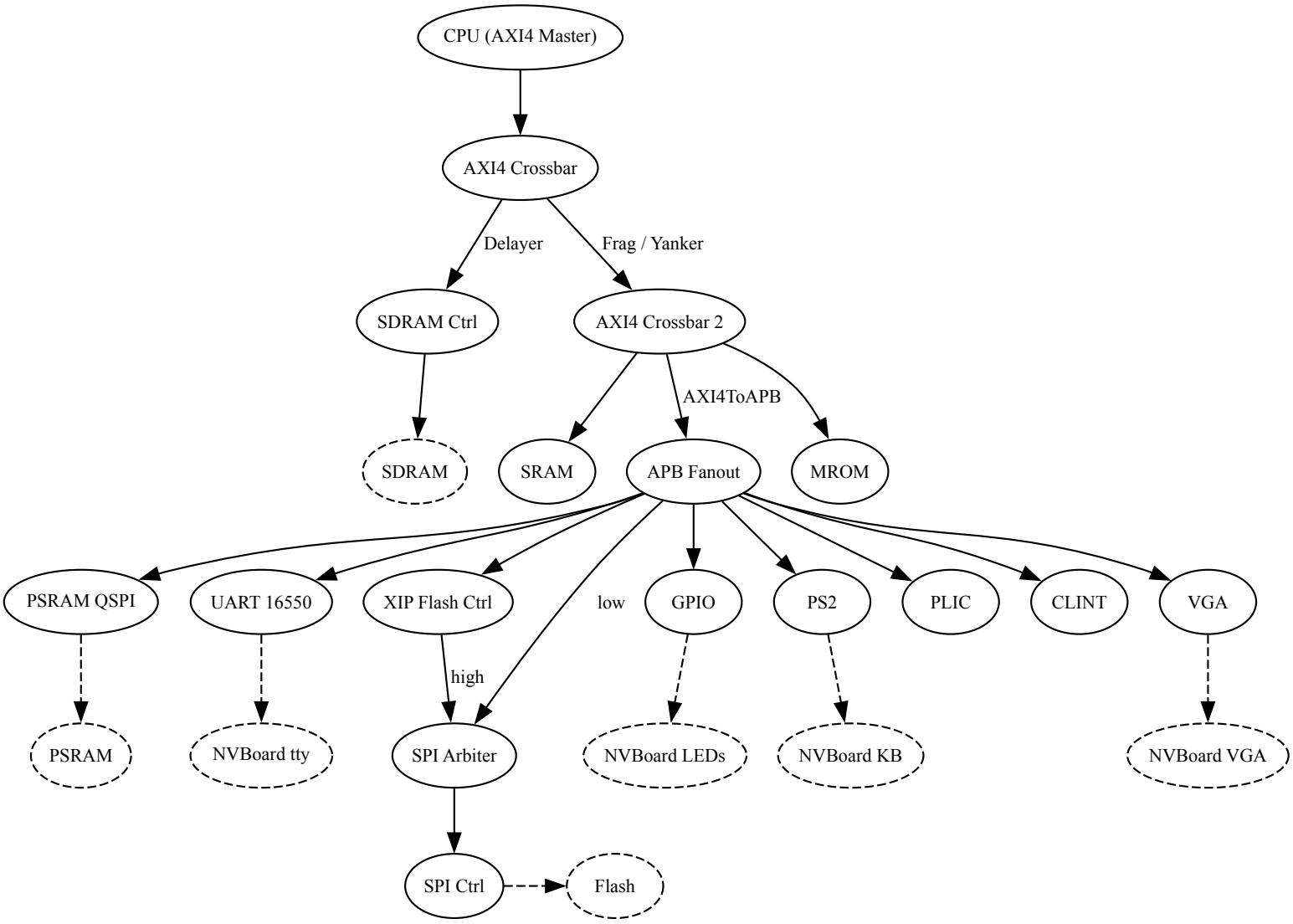

图 2-1　SoC 系统整体互连架构

如 图 2-1 所示，SoC 整体互连分为三个速率域。处理器核心作为唯一的 AXI4 Master 接入一级 AXI4 Crossbar。一级 Crossbar 的左侧分支经 AXI4 Delayer（用于在仿真中注入可配置的访存延迟）连接 SDRAM Controller，右侧分支经 AXI4 Fragmenter（将突发传输拆分为单拍事务以适配不支持 Burst 的从设备）与 AXI4 UserYanker（剥离用户自定义信号以对齐接口宽度）连接二级 AXI4 Crossbar。SDRAM 直接挂载在一级 Crossbar 上，以最短路径获得最高带宽。

二级 Crossbar 下挂 MROM（存放 FSBL, first stage boot loader）、SRAM（低延迟片上存储）以及 AXI4ToAPB 总线桥。总线桥将 AXI4 事务转换为 APB 协议后，由 APB Fanout 按地址空间分发至各低速外设：XIP Flash Controller、PSRAM QSPI Master、UART 16550、GPIO、PS2 键盘控制器、VGA 显示控制器，以及 PLIC（外部中断控制器）和 CLINT（定时器与软件中断控制器）。

在 Flash 访问路径上，XIP Flash Controller 与 APB Fanout 共享底层 SPI Controller 的物理访问权，通过 SPI Arbiter 进行仲裁：XIP 以高优先级发起 SPI 事务用于直接取指执行，APB 以低优先级提供 SPI 寄存器的直接读写通道。各外设控制器的 TX/RX 或数据端口连接至 NVBoard 虚拟开发板对应的外设仿真（串口终端、LED/数码管、键盘、VGA 显示器），以虚线表示。

该分层设计的关键收益在于：其一，高速域与低速域解耦，SDRAM 路径不受慢设备干扰；其二，AXI4 突发与并发事务能力被主存路径充分利用，APB 侧保持实现简单、硬件开销低；其三，桥接与仲裁逻辑集中管理，扩展新外设只需在 APB Fanout 中追加地址映射。

2.3 多层次存储与启动链路设计

存储系统采用 MROM、SRAM、Flash、PSRAM、SDRAM 的多层次组织。MROM 用于放置最小启动代码，SRAM 提供低延迟片上存储，Flash 提供非易失程序镜像并支持 XIP，PSRAM 与 SDRAM 提供更大容量主存。不同存储介质按“启动可靠性—访问延迟—容量”进行功能分工，形成从上电到系统运行的连续存储支撑。

在控制器实现上，关键 IP 核采用 Chisel[19] 实现或重写：SDRAM 控制器面向 AXI4 高带宽路径进行流水化组织，Flash/PSRAM 相关控制器面向 SPI/QSPI/QPI 协议兼容设计。Flash 访问同时支持寄存器方式与 XIP 方式，并通过仲裁机制共享底层 SPI 资源。该设计兼顾了启动阶段可执行性与运行阶段访存效率。

启动链路采用分级引导思路：上电后从 Flash 中的引导入口执行，完成基础初始化后将后续引导或系统镜像加载至 SDRAM，再跳转进入运行态。该流程减少了直接从串行 Flash 执行大体量程序的性能损失，为操作系统（如 xv6[22]、RT-Thread[24]）启动提供更稳定的执行环境。

2.4 外设集成与可视化验证思路

外设侧围绕“可用、可测、可观测”进行设计。UART、GPIO、PS2、VGA 等控制器通过 APB 域接入，保证接口一致性与调试便利性。针对教学与开发场景，系统进一步集成 NVBoard，将串口回显、按键输入、数码管显示与 VGA 输出纳入同一仿真闭环，使 SoC 级功能验证从单纯日志比对扩展为可交互的行为验证。

在验证方法上，SoC 级联调与指令级差分测试协同使用：差分测试用于保证体系结构状态正确性，NVBoard 与外设交互用于暴露系统级时序和接口问题，二者互补提升了复杂问题定位效率。基于该思路，系统能够稳定支撑操作系统运行与外设协同工作。

2.5 本章小结

本章从目标约束、总线分层、存储组织、启动链路与外设验证五个方面阐述了 SoC 设计思路。通过 AXI4/APB 分层互连、多层次存储分工、关键控制器 Chisel 化实现以及 NVBoard 可视化联调，系统在性能、可维护性与可验证性之间取得了平衡，为乱序处理器运行 xv6 等操作系统提供了完整平台基础。

第 3 章　关键 IP 核设计与实现

SoC 系统中的存储控制器直接影响系统启动流程与运行期访存性能。本章围绕 Flash 控制器、PSRAM QSPI Master 控制器与 SDRAM 控制器三个关键 IP 核，分别阐述设计方案与验证方法。

3.1　Flash 控制器与 XIP 机制

Flash 存储器通过 SPI 协议接入系统。SPI Master 选用 OpenCores 开源 IP 核，基于 Wishbone 总线接口，本课题使用 Chisel 实现了 APB-to-Wishbone 的薄封装层以接入 SoC 的 APB 域。

系统为 Flash 提供两种访问方式。第一种是命令方式：通过 APB Fanout 经 SPI Arbiter 直接读写 SPI 控制器寄存器，适用于 Bootloader 阶段的 Flash 编程与配置操作。第二种是 XIP（eXecute In Place）方式：XIP 控制器将 Flash 地址空间映射到处理器的统一地址空间，自动将 CPU 的存储器读请求转换为 SPI 读命令，使处理器可直接从 Flash 取指执行。两种方式通过 SPI Arbiter 共享物理 SPI 控制器，XIP 具有更高的仲裁优先级。

在仿真验证方面，参照 Winbond W25Q128JVSIQ 手册中的时序与命令字规范，实现了 Flash 芯片的行为级仿真模型，并通过类 UVM 的验证测试确保协议兼容性。该仿真模型的协议正确性意味着上板或流片时可直接对接物理 Flash 芯片。

3.2　PSRAM QSPI Master 控制器

PSRAM 通过 QSPI/QPI 协议接入系统，用于提供主存扩展。本课题参考 OpenCores SPI 设计思路，使用 Chisel 重新实现了完整的 QSPI Master 控制器，采用 APB 接口，支持从标准 QSPI 模式切换到 QPI 模式（四线半双工）。

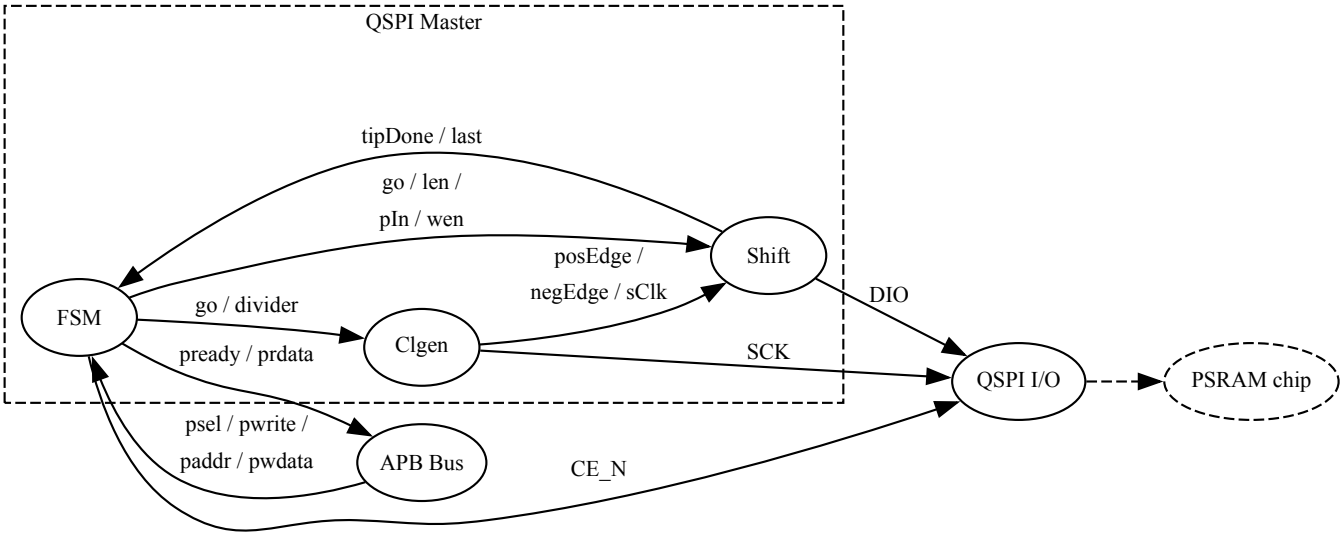

图 3-1　QSPI Master 控制器内部架构

如图 3-1 所示，控制器内部由三个子模块构成：状态机（FSM）负责协调 APB 总线事务与 SPI 传输流程；时钟分频器（Clgen）根据配置的分频系数生成 SPI 时钟，并向移位寄存器提供采样与驱动边沿信号；移位寄存器（Shift）在 Clgen 提供的时钟边沿驱动下完成数据的串并转换。FSM 通过 go 信号启动一次传输，Shift 在传输完成后通过 tipDone/last 信号通知 FSM，FSM 随即向 APB 侧返回 pready 完成握手。

在仿真验证方面，参照 ISSI IS66WVS4M8ALL/BLL 手册中的命令字与时序规范，实现了 PSRAM 颗粒的行为级仿真模型，用于在 Verilator 仿真环境中验证 QSPI 控制器的协议正确性。

3.3　SDRAM 控制器

为提供较大容量的主存储空间，SDRAM 控制器挂载在一级 AXI4 Crossbar 上。本课题参考 ultraembedded 开源的 SDRAM 控制器（Verilog 实现），使用 Chisel 进行了完整重写，以实现参数化扩展并与 SoC 其他 Chisel 模块统一维护。重写后的控制器利用 Chisel 内置的 Queue 模块对请求和响应通路进行解耦，流水化程度高于原始 Verilog 版本。

控制器支持位扩展与字扩展。位扩展模式下，单颗粒提供 16bit 数据宽度，双颗粒拼接为 32bit。字扩展模式下，两个 SDRAM Core 以低位交叉（Low-order Interleaving）方式并联，分别映射到地址的奇偶字。

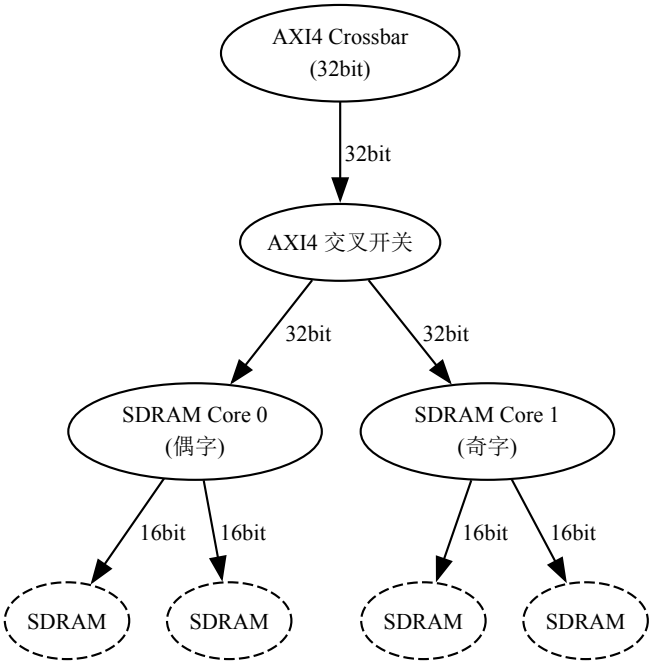

图 3-2　SDRAM 控制器字扩展与低位交叉架构

如 图 3-2 所示，AXI4 交叉开关将连续地址的访问分发到不同 Core，使两颗粒可并行处理不同地址的请求，提高访存并发度。

在仿真验证方面，参照 Micron MT48LC16M16A2 手册实现了 SDRAM 颗粒的行为级仿真模型，准确模拟了 ACTIVATE、READ、WRITE、PRECHARGE、AUTO REFRESH 等命令的时序行为。验证平台采用基于 SystemC 的 UVM 架构，整体框图如 图 3-3 所示。

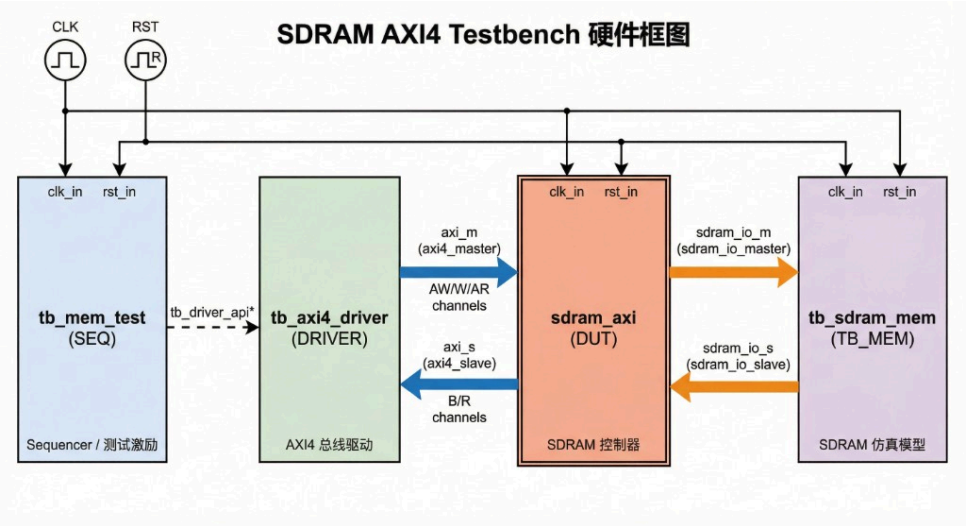

图 3-3　SDRAM AXI4 Testbench 硬件框图

该测试平台由四个模块组成。tb_mem_test（SEQ）作为 Sequencer 负责产生测试激励，包括随机地址、数据长度与突发模式等参数，并通过 tb_driver_api 接口传递给 tb_axi4_driver（DRIVER）。tb_axi4_driver 将测试激励转换为符合 AXI4 协议的总线事务，通过 axi_m（axi4_master）接口向 DUT 发起 AW/W/AR 通道请求，并从 axi_s（axi4_slave）接口接收 B/R 通道响应。sdram_axi（DUT）即待测的 SDRAM 控制器，接收 AXI4 事务并将其转换为 SDRAM 物理接口时序，通过 sdram_io_m（sdram_io_master）接口驱动下游存储模型。tb_sdram_mem（TB_MEM）为 SDRAM 芯片的行为级仿真模型，通过 sdram_io_s（sdram_io_slave）接口响应控制器发出的命令序列。全部模块共享统一的时钟（CLK）与复位（RST）信号。本课题在该平台基础上进行了简单重构，以支持 outstanding 与乱序响应场景的验证，最终确认了 SDRAM 控制器的协议正确性。

3.4 本章小结

本章介绍了 SoC 中三个关键存储控制器的设计与验证。Flash 控制器通过 SPI Arbiter 同时支持命令访问与 XIP 直接取指执行；PSRAM QSPI Master 以 FSM + Clgen + Shift 三模块架构实现 QSPI/QPI 协议兼容；SDRAM 控制器通过 Chisel 重写实现了流水化与低位交叉字扩展。三个控制器均配有行为级仿真模型并通过协议级验证，具备直接对接物理芯片的能力。

第 4 章　前后端解耦架构

4.1　前后端解耦架构设计

本项目实现的乱序处理器采用前后端解耦架构。前端（FrontEnd）负责取指与分支预测，后端（BackEnd）负责译码、寄存器重命名、分派、发射、执行、写回、晚执行、提交。前后端之间通过一个指令队列（Instruction Queue）进行缓冲与解耦，后端在需要冲刷流水线时通过 flush 信号同步清空该队列，分支预测失败或异常恢复时通过 redirect 信号将正确的 PC 反馈给前端。

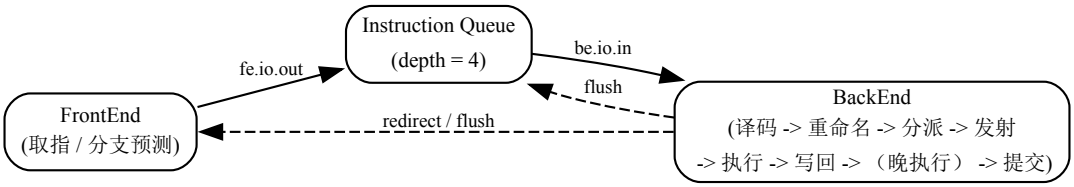

图 4-1　前后端解耦架构与指令队列

如 图 4-1 所示，前端产生的指令流经指令队列缓冲后进入后端；后端在检测到分支预测错误或异常时，通过 flush 信号清空队列中的无效指令，同时通过 redirect 信号将恢复后的目标 PC 发送给前端，使前端从正确地址重新开始取指。

4.2　前后端解耦架构的含义

前后端解耦的含义体现在时序解耦与实现解耦两个层面，并由此带来弹性的设计空间。

在时序解耦方面，前后端之间的连接方式是可配置的：既可以选择点对点直连（组合逻辑穿透，零周期延迟），也可以插入深度可调的缓冲队列。当使用队列时，前后端各自按自身节奏运行——后端因数据依赖或功能单元冲突暂停消费时，前端仍可继续取指填充队列，避免取指带宽浪费；前端遭遇 ICache 缺失时，后端可继续执行队列中已缓冲的指令，减少流水线气泡。队列深度作为参数，可根据实际时序需求灵活调整。

在实现解耦方面，前端与后端通过标准握手协议通信，两侧的内部实现完全独立。前端既可以采用流水化结构，也可以采用状态机驱动，甚至可以完全使用软件实现。在开发过程中，硬件前端由于需要等待总线响应与 ICache 缺失处理，指令供应速率有限，后端负载偏低，难以充分暴露乱序流水线在高压力下的时序问题。为此，本课题利用解耦架构的可替换性，在前端内嵌入 Spike 参考模型：Spike 每执行一条指令即可获得精确的下一条指令 PC，使分支预测几乎 100% 正确，同时取指不经过总线，几乎每个周期都能向后端供应一条有效指令。这一做法相当于对后端进行了满载压力测试，成功暴露了一个在常规硬件前端下因指令供应不足、未形成反压（back-pressure）而被隐藏的 bug——该 bug 表现为反压条件下数据转发时机错过，仅在后端持续满载时才会触发。定位并修复该问题后，前端切换回正常硬

件实现。图 4-2 展示了使用 Spike 软件前端运行 MicroBench 测试套件的结果：全部测试项通过，共提交 624375 条指令，其中 142230 条分支指令仅产生 514 次预测错误，命中率达 99.64%，验证了该方案的指令供应充分性。这一经验表明，解耦架构不仅降低了模块间的开发耦合度，还为针对性的压力测试提供了灵活的验证手段。

```
======= Running MicroBench [input *test*] =======
[qsort] Quick sort: * Passed.
[queen] Queen placement: * Passed.
[bf] Brainf**k interpreter: * Passed.
[fib] Fibonacci number: * Passed.
[sieve] Eratosthenes sieve: * Passed.
[15pz] A* 15-puzzle search: * Passed.
[2026-04-04T07:11:14Z INFO  npc::common::lightsss] [lightsss] forked checkpoint pid=57228, slots=2
[dinic] Dinic's maxflow algorithm: * Passed.
[lzip] Lzip compression: * Passed.
[ssort] Suffix sort: * Passed.
[md5] MD5 digest: * Passed.
==================================================
MicroBench PASS
Scored time: 0.000 ms
Total  time: 0.000 ms
[2026-04-04T07:11:15Z INFO  npc::libsdb::sdb] program exited successfully
[2026-04-04T07:11:15Z INFO  npc] commit: 624375
[2026-04-04T07:11:15Z INFO  npc] branch: 142230
[2026-04-04T07:11:15Z INFO  npc] mispredict: 514
[2026-04-04T07:11:15Z INFO  npc] hit rate: 0.9963861
[2026-04-04T07:11:15Z INFO  npc] flush: 514
make[1]: Leaving directory '/Users/wangfiox/Documents/ysyx/ysyx-workbench/npc'
```

图 4-2　Spike 软件前端运行 MicroBench 的测试结果（分支预测命中率 99.64%）

4.3　本章小结

弹性设计空间是解耦架构的另一直接收益。指令队列作为缓冲层，使得前端分支预测器的升级（如从取指状态机切换到取指流水线）不影响后端接口，有利于微架构的增量迭代与独立优化。

第 5 章　前端：取指单元与动态分支预测设计

前端负责为后端持续供应有效指令，其内部由取指单元（IFU）与分支预测单元（Predict）两个子模块组成。IFU 通过状态机驱动完成地址翻译与 ICache 访问，Predict 对当前取到的指令进行轻量译码并输出预测的下一条指令 PC，两者协同工作形成“取指—预测—下一轮取指”的闭环。

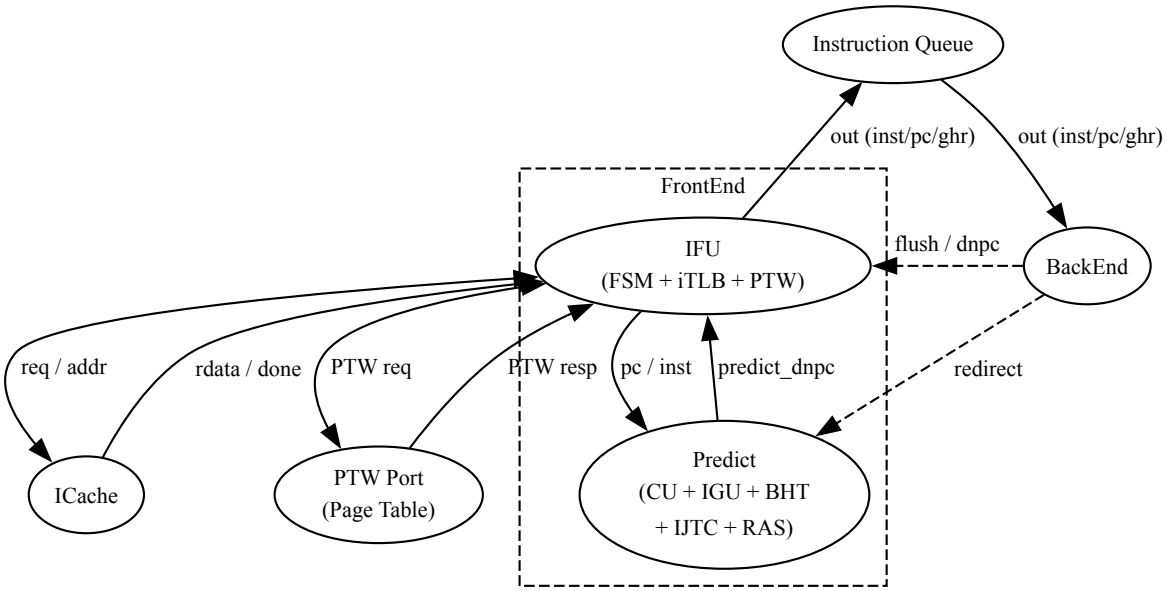

图 5-1　前端内部架构与外部接口

如 图 5-1 所示，IFU 通过 ICache 端口和 PTW(page table walker) 端口分别访问指令缓存与页表，取到的指令与 PC 传递给 Predict 进行分支预测，Predict 返回预测的下一条 PC 供 IFU 在下一轮取指时使用。IFU 的输出（指令、PC、预测信息）经指令队列送入后端。后端在分支预测失败或异常恢复时，通过 redirect 信号更新 Predict 的预测器状态，并通过 flush/dnpc 信号使 IFU 从正确地址重新取指。

5.1　取指单元（IFU）状态机设计

IFU 采用七状态有限状态机驱动取指流程，以适配地址翻译与 ICache 访问中的不定长延迟。

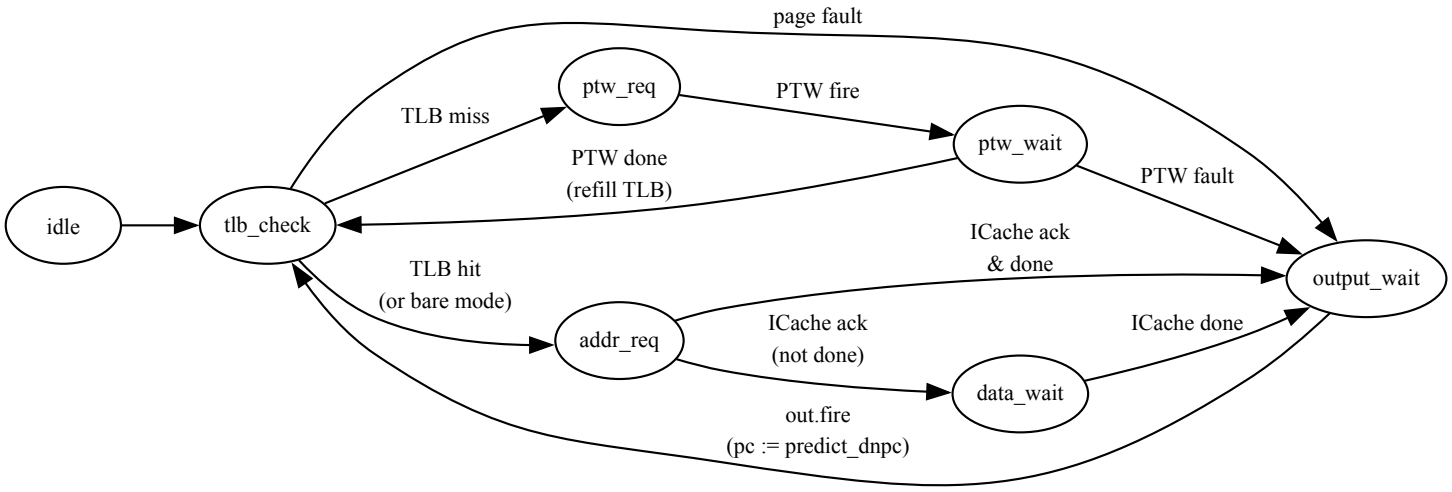

图 5-2　IFU 取指状态机

如 图 5-2 所示，上电后状态机从 idle 无条件进入 tlb_check。在 tlb_check 状态，若 Sv39 未启用则直接使用物理地址进入 addr_req；若 Sv39 已启用且 iTLB 命中，则检查页表项的执行权限位（PTE.X），权限合法则进入 addr_req，否则标记 page fault 进入 output_wait；若 iTLB 缺失，则进入 ptw_req 启动页表遍历。PTW 完成后回填 iTLB 并回到 tlb_check 重新查询；若 PTW 检测到页错误，则直接进入 output_wait 报告异常。

addr_req 状态向 ICache 发起访问请求。若 ICache 单周期命中（ack 且 done 同时有效），直接进入 output_wait；否则进入 data_wait 等待缓存行填充完成。在 output_wait 状态，IFU 将取到的指令、PC 与预测信息置于输出端口，等待下游握手。当 out.fire 有效时，PC 寄存器更新为 Predict 输出的预测地址，状态机回到 tlb_check 开始下一轮取指。flush 信号可在任意状态强制将状态机复位至 tlb_check 并加载后端提供的恢复 PC。

5.2　动态分支预测单元设计

分支预测单元（Predict）对 IFU 取到的指令进行轻量译码，复用后端的 CU（Control Unit，控制单元）与 IGU（Immediate Generate Unit，立即数生成单元）模块提取指令类型与立即数。CU 本身是后端译码级的核心模块，包含完整的指令解码逻辑，具有一定的复杂性。前端复用 CU 时，将前端不涉及的输入信号硬编码为 0，不需要的输出信号悬空；综合工具在优化阶段可据此自动裁剪不可达的组合逻辑路径，使前端实例化的 CU 仅保留指令类型识别等轻量子集，在避免重复编写译码逻辑的同时不引入多余的面积开销。Predict 根据 CU 输出的指令类型将当前指令分派到不同的预测器，最终通过优先级选择逻辑输出预测的下一条 PC。

5.2.1 GShare 分支方向预测器（BHT）

对于条件分支指令，采用 GShare 算法[17] 进行方向预测。GShare 通过全局历史寄存器（GHR）与 PC 的异或结果索引模式历史表（PHT），利用全局分支相关性提高预测精度。PHT 包含 4096 个表项，每个表项使用 2 位饱和计数器：计数器高位为 1 表示预测跳转，为 0 表示预测不跳转。当 PHT 中对应表项尚未被训练（occupied 标志为假）时，退化为静态预测——以立即数符号位作为预测方向（负偏移预测跳转，正偏移预测不跳转），该启发式对循环结构有较好的覆盖。

后端在条件分支指令提交时，通过 redirect 信号将实际跳转结果与该指令对应的 GHR 快照回传给 BHT，BHT 使用回传的 GHR 与 PC 异或重新定位 PHT 表项并更新饱和计数器，同时将实际跳转方向移入 GHR。

5.2.2 间接跳转目标缓存（IJTC）

对于非返回类的间接跳转指令（JALR 且非 ret 模式），采用 IJTC 进行目标地址预测。IJTC 使用 8 项全相联 CAM 结构，以 PC 作为 tag 进行匹配查询。命中时直接返回缓存的目标地址；缺失时不做预测，前端将使用顺序 PC。

更新策略上，后端在 JALR 指令提交时将实际 PC 与目标地址回传给 IJTC。若 CAM 中已存在该 PC 对应的表项则原地更新目标地址；若不存在则以 FIFO 策略淘汰最旧表项并写入新映射。

5.2.3 返回地址栈（RAS）

对于函数返回指令，采用 RAS 进行目标地址预测。RAS 为深度 8 的循环栈，通过 rd 和 rs1 字段识别 call/ret 模式：当指令为 JAL 或 JALR 且目的寄存器为 x1 或 x5 时判定为 call，将顺序 PC（snpc = pc + 4）压栈；当指令为 JALR 且 rd = x0、rs1 = x1 时判定为 ret，弹栈并将栈顶值作为预测目标。

5.2.4 预测选择逻辑

Predict 模块根据指令类型按优先级选择预测结果：JAL 指令直接使用 PC + 立即数作为目标；条件分支指令根据 BHT 预测方向选择跳转目标或顺序 PC；JALR 且为 ret 模式时使用 RAS 弹栈结果；JALR 且 IJTC 命中时使用 IJTC 缓存的目标地址；其余情况使用顺序 PC。预测结果与当前 GHR 快照一并传递给 IFU，由 IFU 在下一周期使用该地址取指，并随指令一同送入后端，供后端在提交时进行验证与预测器更新。

5.2.5 与 BTB 方案的对比及设计取舍

经典分支预测方案[11] 通常使用分支目标缓冲器（BTB, Branch Target Buffer）：在取指阶段以 PC 索引 BTB，若命中则直接读出缓存的跳转目标地址，从而在译码之前就完成预测。BTB 的优势在于无需等待指令译码即可获得目标地址，适合流水化取指前端。

本课题的设计目标是在尽可能简单的前端实现下取得合理的预测覆盖。由于前端采用状态机而非流水线结构，每条指令在 output_wait 阶段已完成取指与预解码，指令类型和立即数均可获得。对于 JAL 和 BRANCH 这两类指令，目标地址

为 PC + 立即数的简单加法，可在当拍直接计算得出，无需额外的缓存结构；仅对 JALR 这类目标地址依赖寄存器值、前端无法计算的指令，才借助 IJTC 或 RAS 进行预测。因此在当前微架构下，BTB 的作用被直接计算和专用预测器所替代，省去了 BTB 的存储开销与维护逻辑。若后续前端演进为流水化结构，BTB 将成为必要的补充。

5.2.6 预测器更新策略

三个预测器的更新策略根据各自特性采用了不同的时机。

GShare 与 IJTC 采用保守策略，仅在指令提交（commit）时更新。后端通过 redirect 信号将已提交的分支指令的实际跳转结果、PC 与 GHR 快照回传给前端预测器。这一策略的优势在于只有确认正确执行路径上的指令才会影响预测器状态，避免了因推测执行路径上的错误信息污染 PHT 或 IJTC 表项。

RAS 则采用激进策略，在前端取指与预解码阶段即时更新：遇到 call 指令立即压栈，遇到 ret 指令立即弹栈。这一设计是合理的——即使当前指令处于推测路径上，推测路径中的 call 与 ret 仍然是成对出现的，压栈与弹栈操作相互抵消，不会破坏栈的正确性。若在前端就更新 RAS，后续紧跟的 ret 指令可以立即获得正确的返回地址，而无需等待 call 指令提交，从而降低了函数调用密集场景下的预测延迟。

5.3 本章小结

本章介绍了前端的取指单元与动态分支预测设计。IFU 采用七状态 FSM 适配 Sv39 地址翻译与 ICache 不定长延迟；分支预测采用 GShare + IJTC + RAS 组合方案，分别覆盖条件分支、间接跳转与函数返回三类控制流场景。在目标地址获取上，利用前端已完成预解码的特点对 JAL 与 BRANCH 直接计算目标地址，仅对 JALR 依赖专用预测器，省去了 BTB 开销。在更新策略上，GShare 与 IJTC 保守地在指令提交时更新以避免推测污染，RAS 则激进地在前端即时更新以利用 call/ret 配对特性降低预测延迟。在 MicroBench 测试套件 test 数据规模下（如 图 5-3 所示），处理器共提交 653614 条指令，其中 156330 条分支指令产生 20263 次预测错误，分支预测命中率达到 87.0%，验证了 GShare + IJTC + RAS 组合方案在实际工作负载下的有效性。

```
======= Running MicroBench [input *test*] =======
[qsort] Quick sort: * Passed.
[queen] Queen placement: * Passed.
[bf] Brainf**k interpreter: * Passed.
[fib] Fibonacci number: * Passed.
[sieve] Eratosthenes sieve: * Passed.
[15pz] A* 15-puzzle search: * Passed.
[dinic] Dinic's maxflow algorithm: * Passed.
[lzip] Lzip compression: * Passed.
[ssort] Suffix sort: * Passed.
[md5] MD5 digest: * Passed.
==================================================
MicroBench PASS
Scored time: 1407.014 ms
Total  time: 1867.998 ms
I0413 12:17:15.497395 1558507 sdb.cc:117] program exited successfully
I0413 12:17:15.497462 1558507 main.cc:74] commit: 653614
I0413 12:17:15.497470 1558507 main.cc:75] branch: 156330
I0413 12:17:15.497471 1558507 main.cc:76] mispredict: 20263
I0413 12:17:15.497473 1558507 main.cc:77] hit rate: 0.870383
I0413 12:17:15.497476 1558507 main.cc:78] flush: 20264
make[1]: Leaving directory '/Users/wangfiox/Documents/ysyx/ysyx-workbench/npc'
```

图 5-3 MicroBench test 数据规模下的分支预测统计（命中率 87.0%）

第 6 章　后端：乱序执行引擎

后端接收前端经指令队列送入的指令流，完成译码、寄存器重命名、发射、执行、写回与提交的全流程，是实现乱序执行的核心引擎[12]。本项目实现的后端支持 RV64IM 指令集架构，包含整数运算、乘除法运算、分支、访存与 CSR 五条执行路径。

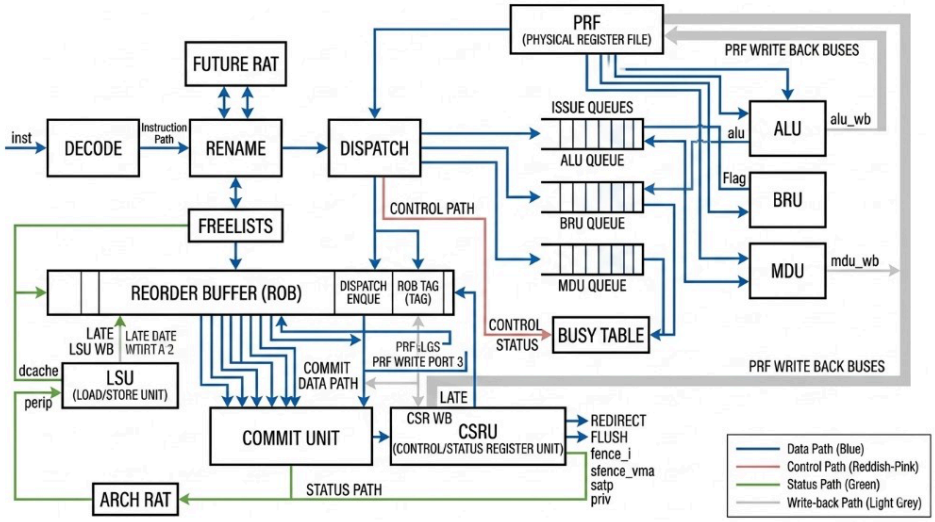

图 6-1　后端乱序执行引擎（由 Nano Banana2 模型根据手绘草图与 Chisel 源码辅助生成）

如 图 6-1 所示，图中蓝色连线表示指令与数据通路，粉色连线表示控制通路（wakeup 与 BusyTable 状态查询），绿色连线表示提交级状态通路（ArchRAT 写回与 FreeList 回收），灰色连线表示 PRF 写回总线。指令从左侧进入 Decode 级，经 Rename 级查询 FutureRAT 并从 FreeList 分配物理寄存器后，进入 Dispatch 级。Dispatch 将指令写入 ROB 获取 rob_tag，同时根据指令类型分派到 ALU Queue、BRU Queue 或 MDU Queue 三个发射队列。发射队列通过 BusyTable 查询源操作数就绪状态，就绪后发射到对应执行单元：ALU 与 MDU 执行后将结果通过 PRF 写回总线写入 PRF 并向 ROB 报告完成，BRU 仅向 ROB 报告分支结果（Flag）。LSU 与 CSRU 采用延迟执行路径，由 ROB 在指令接近提交时交付执行，LSU 连接 DCache 与外设端口，CSRU 负责 CSR 读写并输出 redirect、flush、fence_i、sfence_vma、satp 与 priv 等控制信号。提交级（Commit Unit）从 ROB[25] 头部按序提交，将新映射写入 ArchRAT、旧物理寄存器归还 FreeList，完成乱序执行到顺序提交的状态收敛[13]。

6.1　译码级

译码级接收前端送入的指令字与 PC，通过 CU（Control Unit）解码出指令类型、功能单元选择、操作码、源/目的寄存器编号等控制信号，通过 IGU（Immediate Generate Unit）提取并符号扩展立即数。译码级的输出形成完整的微操作描述，经流水线寄存器传递给重命名级。

CU 的实现利用了 Chisel 提供的 `DecodeTable` 框架与 Espresso 逻辑最小化工具。具体而言，首先定义 `InstPattern` 作为指令编码的匹配模板，将 func7、rs2、func3、opcode 四个字段组合为 `BitPat`，其中不相关的位标记为 don't-care：

```
case class InstPattern(
  func7: BitPat = BitPat.dontCare(7),
  rs2:   BitPat = BitPat.dontCare(5),
  func3: BitPat = BitPat.dontCare(3),
  opcode: BitPat
) extends DecodePattern {
  def bitPat = func7 ## rs2 ## BitPat.dontCare(5)
             ## func3 ## BitPat.dontCare(5) ## opcode
}
```

代码 6-1　指令匹配模板 InstPattern 定义

随后，为每个需要译出的控制信号定义一个 `DecodeField`，在其 `genTable` 方法中以 Scala 模式匹配的方式描述每条指令对应的输出值。所有指令模板与译码字段汇总为一张 `DecodeTable`，Chisel 编译期自动调用 Espresso 对真值表进行两级逻辑最小化，生成面积最优的组合逻辑电路。相比手工编写 `MuxLookup` 或 `switch` 译码树，该方案在指令数量增长时仍能保持紧凑的门级实现，同时 Scala 的类型系统与模式匹配语法使译码表的维护更加直观，新增指令只需在 `allInstructions` 列表中追加一行 `InstPattern` 并在各 `DecodeField` 中补充对应映射即可。

CU 的译码输出涵盖 9 个字段：指令类型（`InstType`，18 种）、ALU 操作码（`ALUOpType`）、MDU 操作码（`MDUOpType`）、立即数类型（`ImmType`，I/S/B/U/J 五种）、BRU 操作码（`BRUOpType`）、访存宽度与符号扩展标志、CSR 操作类型与写使能。对于 ECALL、EBREAK 与非法指令，CU 额外输出异常标记与 mcause/mtval 初始值，供提交级进一步处理。

6.2　寄存器重命名

寄存器重命名是乱序执行的核心前提[26]，其目的是消除指令间由逻辑寄存器复用导致的 WAW（写后写）与 WAR（写后读）假依赖，使真正的数据依赖（RAW）成为唯一的调度约束。本项目采用统一物理寄存器文件（Unified Physical Register File, PRF）方案，主要涉及以下结构：

物理寄存器文件（PRF）。PRF 提供 9 个读端口与 5 个写端口。9 个读端口分配给 ALU（2）、BRU（2）、MDU（2）、LSU（2）与 CSRU（1）五条执行路径；5 个写端口分别服务于 ALU 写回、分派级立即数写入、LSU 写回、CSR 写回与 MDU 写回。

Future RAT。Future RAT 维护推测状态下的逻辑寄存器到物理寄存器映射表。重命名级通过 3 个读端口查询源寄存器（rs1、rs2）与目的寄存器（rd）当前映射的物理寄存器号，随后从 FreeList 分配一个空闲物理寄存器作为 rd 的新映射，并将新映射写回 Future RAT。旧的 rd 映射（old_prd）随指令一同写入 ROB，供提交时释放。

Arch RAT。Arch RAT 维护已提交的确定性映射。每当指令提交时，CommitStage 将该指令的逻辑目的寄存器与新物理寄存器映射写入 Arch RAT，并将 old_prd 归还 FreeList。Arch RAT 始终反映最近一次已提交指令的寄存器状态。

FreeList。FreeList 管理空闲物理寄存器池。重命名级从中分配新物理寄存器，提交级向其归还不再需要的物理寄存器。归还的依据是：当一个逻辑寄存器被后续指令重新定义（即发生 WAW）时，承载旧值的物理寄存器的生命周期随之结束，可以被安全释放。以如下指令序列为例：

指令	逻辑 rd	分配 prd	old_prd
ADD x1, x2, x3	x1	p10	p1
SUB x1, x4, x5	x1	p11	p10

ADD 将 x1 的映射从 p1 更新为 p10，此时 p1 成为 old_prd。当 ADD 提交时，p1 承载的旧值已无任何指令需要读取（后续所有引用 x1 的指令都将读到 p10 或更新的映射），因此 p1 可以归还 FreeList。同理，当 SUB 提交时，x1 的映射再次更新为 p11，p10 的生命周期结束并被归还。通过这一机制，物理寄存器的分配与回收严格跟随逻辑寄存器的定义链，保证了在任意时刻每个“活跃值”都独占一个物理寄存器，不会出现数据覆盖。当发生 flush 时，FreeList 根据 Arch RAT 快照重建：凡不在 Arch RAT 映射中的物理寄存器均标记为空闲，一拍即可完成恢复。

BusyTable。BusyTable 记录每个物理寄存器的数据就绪状态。重命名级在分配新物理寄存器时将其标记为 busy；当结果产生时通过 wakeup 信号清除 busy 标记。发射队列在发射前查询 BusyTable 以判断源操作数是否就绪。BusyTable 配置 6 个读端口供三个发射队列使用（每队列 2 个，对应 rs1 和 rs2），5 个 wakeup 端口对应三类时机：（1）分派级即时唤醒——LUI、AUIPC、JAL、JALR 等指令的结果仅依赖 PC 与立即数，在分派级即可计算并写入 PRF，同时发出 wakeup；（2）执行完毕唤醒——ALU 与 MDU 执行单元写回结果时发出 wakeup；（3）延迟执行完毕唤醒——LSU 与 CSRU 在 ROB 调度下完成延迟执行并写回 PRF 后发出 wakeup。三类唤醒覆盖了指令生命周期中所有可能产生结果的阶段，使后续依赖指令能尽早感知操作数就绪并被发射。

Flush 恢复机制。当后端检测到分支预测错误或异常时，flush 信号同时送往 ROB、FreeList、BusyTable、FutureRAT 以及所有发射队列。FutureRAT 从 Arch RAT 的快照一拍恢复到已提交状态的映射；FreeList 同样根据 Arch RAT 快照重建空闲池。由于 Arch RAT 在提交时同步更新，其内容始终正确，因此恢复过程无需遍历 ROB，仅需一个周期。

6.3 分派级

分派级位于重命名级之后，负责将已完成寄存器重命名的微操作分发到对应的执行资源。分派级执行以下操作：

1. 将指令写入 ROB，获取 ROB 标签（rob_tag）作为指令在乱序流水线中的唯一标识。

2. 根据指令类型将指令分派到对应的发射队列：ALU 类指令进入 ALU IQ、乘除法指令进入 MDU IQ、 分支指令进入 BRU IQ。

3. 对于结果可在分派级直接确定的指令——JAL、JALR、LUI 与 AUIPC——分派级直接将计算结果写入 PRF 并发出 wakeup 信号，无需经过执行单元。这些指令的结果（链接地址 pc+4 或立即数加载值）仅依赖 PC 与立即数，在译码后即可获得，提前写回可减少后续依赖指令的等待周期。

4. 对于访存指令与 CSR 指令，采用保守的延迟执行策略：分派级仅将其写入 ROB，不送入发射队列，待指令到达 ROB 提交头部时再交由 LSU 或 CSRU 执行。访存指令的延迟执行避免了推测路径上的 Store 对内存状态造成不可撤销的副作用，简化了存储一致性维护；CSR 指令的延迟执行保证了特权级切换、中断使能等副作用严格按程序顺序生效，防止推测执行引发错误的系统状态变更。

6.4 发射队列与乱序发射

后端设置三个独立的发射队列，分别服务于 ALU、MDU 和 BRU 执行单元。在实现层面，得益于 Chisel 作为 Scala 嵌入式 DSL 的语言特性，发射队列采用泛型抽象类设计：

```
abstract class IssueQueue[T <: Data]
  (gen: T, val entries: Int)
  extends NPCModule {
    ...
  }
```

代码 6-2　泛型发射队列抽象类定义

三个发射队列共享相同的入队、就绪检测与发射选择逻辑，唯一的差异在于类型参数 `gen` 所携带的附加数据——例如 ALU 发射队列的 `gen` 包含立即数与 ALU 操作码字段，而 BRU 和 MDU 的 `gen` 则不含立即数。通过 Scala 的参数化类型，三个队列复用同一套核心实现，仅在类型层面区分载荷内容，避免了重复代码。同时，`abstract class` 的设计允许派生出不同复杂度的具体实现：简单实现采用固定槽位逐项扫描，复杂实现可引入压缩表项（compacting entry）以在指令发射后自动前移后续表项、提高槽位利用率，两者对外接口完全一致，便于在面积与时序之间灵活权衡。

每个发射队列在入队时记录指令的源物理寄存器号与 ROB 标签，在发射前通过 BusyTable 的读端口查询两个源操作数的就绪状态。当源操作数全部就绪时，指令即可被选中发射。由于多条指令可能同时就绪，发射队列内部通过优先级编码器选择最早入队的指令发射，保证公平性。

wakeup 机制是乱序发射的关键驱动力。后端共有 5 个 wakeup 源：ALU 写回、MDU 写回、分派级立即数写入、LSU 写回与 CSRU 写回。当任一执行路径产生结果时，对应的 wakeup 信号广播到 BusyTable，将目的物理寄存器标记为就绪。发射队列在下一周期即可感知新的就绪状态并发射等待该操作数的指令，形成“执行—唤醒—发射”的连锁反应。

6.5 执行单元

后端包含五条执行路径，各路径独立运行，互不阻塞。与发射队列的设计思路类似，执行单元同样抽象出统一的泛型接口：

```
abstract class ExecUnit[I <: Data, O <: Data](
  inGen: I, outGen: O, numReadPorts: Int = 2
) extends NPCModule {
  val prs1 = IO(Input(UInt(NRPhyRegBits.W)))
  val prs2 = IO(Input(UInt(NRPhyRegBits.W)))
  val prf  = IO(Vec(numReadPorts, new PRFReadPort))
  val io = IO(new Bundle {
    val in  = Flipped(Decoupled(inGen))
    val out = Decoupled(outGen)
  })
}
```

代码 6-3　执行单元抽象基类定义

`ExecUnit` 通过类型参数 `I` 和 `O` 分别参数化输入载荷与输出载荷，所有执行单元共享统一的 PRF 读端口接口（`prs1`、`prs2`、`prf`）与 Decoupled 握手协议（`io.in` / `io.out`）。ALU、MDU 与 BRU 继承该抽象类并特化各自的输入输出类型：ALU 的输入包含 ALU 操作码、立即数与立即数选择标志，输出包含运算结果与目的物理寄存器号；MDU 的输入包含 MDU 操作码，输出结构与 ALU 相同；BRU 的输入仅包含分支操作码，输出为分支结果标志（br_flag）。这一设计使后端连线层可以统一处理发射队列到执行单元的对接逻辑，同时各执行单元内部实现完全独立。

ALU 路径。处理整数算术逻辑运算（加减、移位、比较、逻辑运算等）。ALU 为单周期执行，从 ALU IQ 发射后经 PRF 读取源操作数、执行运算、写回 PRF 并向 ROB 报告完成，同时发出 wakeup。

MDU 路径。处理乘法与除法运算。MDU 执行延迟不固定（乘法与除法周期数不同），通过 valid/ready 握手协议与发射队列和 ROB 交互。写回逻辑与 ALU 路径相同。

BRU 路径。处理条件分支指令的实际比较。BRU 从 BRU IQ 发射后读取源操作数并计算分支条件（br_flag），将结果报告给 ROB。BRU 不写 PRF——条件分支指令无目的寄存器。

6.6 延迟执行单元

与通过发射队列乱序发射的 ALU/MDU/BRU 不同，LSU 与 CSRU 采用延迟执行（late execution）模式：指令在分派时仅写入 ROB，待到达 ROB 提交头部时才交付执行。延迟执行单元同样抽象出统一的泛型基类：

```
abstract class LateExecUnit[T <: Data](
  gen: => T, numReadPorts: Int
) extends NPCModule {
  val late = IO(Flipped(ReqDone(gen)))
  val prf  = IO(Vec(numReadPorts, new PRFReadPort))
}
```

代码 6-4　延迟执行单元抽象基类定义

`LateExecUnit` 通过类型参数 `T` 参数化 ROB 传入的请求载荷，`late` 端口采用 `ReqDone` 握手协议（ROB 发起请求、执行单元返回完成），`prf` 提供 PRF 读端口用于读取源操作数。LSU 与 CSRU 继承该基类并特化各自的请求类型与读端口数量。

LSU 路径。处理 Load/Store 指令。ROB 将访存指令送至提交头部后通过 `late` 端口交给 LSU，确保访存操作按程序顺序执行，简化了存储一致性维护。LSU 内部连接 DCache 与外设端口，并集成 dTLB 与 PTW 以支持 Sv39 地址翻译。LSU 写回通过独立的 PRF 写端口与 wakeup 通路完成。

CSRU 路径。处理 CSR 读写指令。与 LSU 类似，由 ROB 在提交阶段通过 `late` 端口交给 CSRU 执行，保证 CSR 操作的顺序语义。CSRU 同时负责中断与异常的状态管理，包括 trap 入口计算、mret/sret 返回地址提供、中断挂起检测等。

6.7　提交级

提交级（CommitStage）从 ROB 头部按程序顺序逐条提交指令，是乱序执行恢复顺序语义的关键环节。提交时执行以下操作：

1. 将指令的目的逻辑寄存器与新物理寄存器映射写入 Arch RAT，使确定性状态前进。

2. 将旧物理寄存器号（old_prd）归还 FreeList，供后续指令重命名时分配。

3. 检查异常与中断：若提交头部的指令携带异常标记（如 page fault、非法指令）或检测到中断挂起，CommitStage 触发 trap 流程，通过 CSRU 获取 trap 入口地址，生成 redirect 信号指向异常处理程序，并发出 flush 清空流水线。

4. 处理 mret/sret 返回：提交 mret 或 sret 指令时，从 CSRU 读取 mepc 或 sepc 作为 redirect 目标。

5. 处理 fence.i 与 sfence.vma：提交这两类指令时，分别向外输出 fence_i 和 sfence_vma 信号，通知 ICache 失效或 TLB 刷新。

6. 分支验证：对于条件分支指令，将 BRU 写回的实际跳转结果与前端预测进行比对。若预测错误，生成 redirect 信号携带正确的目标地址与分支信息，同时发出 flush。

7. 调试探针（probe）：提交时输出指令的 PC、下一条 PC、指令字、GPR 状态与 CSR 状态等信息，供 DiffTest 框架与参考模型逐条比对，实现运行时正确性验证。

6.8　本章小结

本章介绍了后端乱序执行引擎的设计。后端采用译码、重命名、分派、发射、执行、写回、（晚执行）、提交的经典乱序流水线结构，通过统一物理寄存器文件与

FutureRAT/ArchRAT 双映射表实现寄存器重命名，消除 WAW 与 WAR 假依赖。五条独立执行路径（ALU、MDU、BRU、LSU、CSRU）并行运行，其中 ALU/MDU/BRU 通过发射队列乱序发射，LSU 与 CSRU 通过 ROB 延迟执行以保证顺序语义。提交级按序提交并处理异常、中断、分支验证与调试输出。flush 恢复仅需一周期，利用 ArchRAT 快照同时重建 FutureRAT 与 FreeList。

第 7 章　RISC-V 特权级架构

本章介绍处理器的特权级架构、精确异常与中断机制的设计与实现。实现主要参考 RISC-V 特权级规范[27]、SBI 规范[28] 以及 PLIC[29] 与 CLINT 规范手册[30]。

7.1　M/S/U 三级特权架构

处理器实现了 RISC-V 规范定义的三级特权模式：M-mode（机器模式）、S-mode（监管模式）与 U-mode（用户模式）。当前特权级由 CSRU 内部的 `priv` 寄存器维护，上电复位后初始化为 M-mode。特权级的切换由两类事件驱动：

1. 陷入（Trap）：异常或中断发生时，处理器根据委托寄存器（medeleg/mideleg）决定陷入到 M-mode 还是 S-mode，并将当前特权级保存到 mstatus 的 MPP 或 SPP 字段。

2. 返回（xRET）：执行 mret 时从 mstatus.MPP 恢复特权级；执行 sret 时从 mstatus.SPP 恢复特权级。

图 7-1 展示了 M/S/U 三级特权模式之间的完整转换关系。

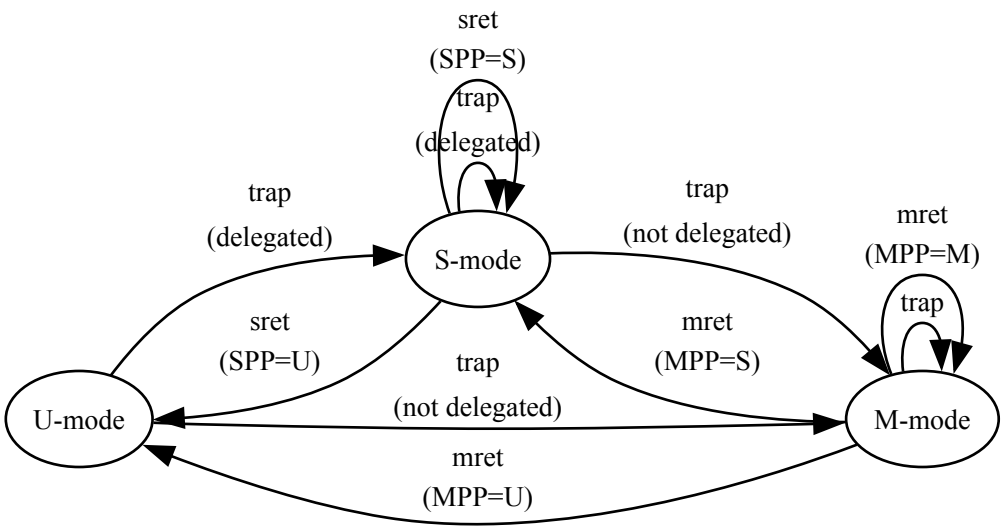

图 7-1　M/S/U 特权级状态机

通过委托(delegate)机制，操作系统可在 S-mode 直接处理用户态的系统调用与页错误等常见异常，无需经过 M-mode 中转，降低了 trap 延迟。

7.2　CSR 寄存器文件

CSRU 内部实现了完整的 M-mode 与 S-mode CSR 寄存器集合：

模式	寄存器	功能
M	mstatus, mtvec, mepc, mcause, mtval	状态、陷入入口、异常现场
M	medeleg, mideleg	异常/中断委托

模式	寄存器	功能
M	mie, mip	中断使能与挂起
M	mscratch, menvcfg, mcounteren	暂存、环境配置、计数器访问
M	pmpcfg0, pmpaddr0	物理内存保护
M	mvendorid, marchid, mhartid, misa	只读标识（RV64IMASU）
S	sstatus, stvec, sepc, scause, stval	S-mode 状态与陷入现场
S	sie, sip, sscratch, satp	中断使能、暂存、页表基址
S	stimecmp	S-mode 定时器比较值
U	time（只读）	读取 mtime

其中 sstatus、sie 与 sip 并非独立寄存器，而是 mstatus、mie 与 mip 的掩码视图（masked view）。例如 sstatus 的读取值为 `mstatus & SSTATUS_MASK`，写入时仅修改掩码对应位，其余位保持不变。这一设计与 RISC-V 规范一致，避免了 M/S 两级寄存器状态的冗余与不一致。

CSR 指令支持三种操作类型：CSRRW（读后写）、CSRRS（读后置位）与 CSRRC（读后清零），写入数据分别为 `wdata`、`csr_read | wdata`、`csr_read & ~wdata`。

7.3 精确异常

本项目通过 ROB 顺序提交机制实现精确异常。异常在流水线各阶段被检测后，仅在对应指令的 ROB 表项中标记异常类型（mcause）与辅助信息（mtval），不立即影响处理器状态。当该指令到达 ROB 提交头部时，CommitStage 检查异常标记并触发 trap 流程：

1. CommitStage 根据当前特权级确定 ECALL 的实际异常号（U-mode → 8，S-mode → 9，M-mode → 11），其他异常类型直接使用译码级标记的 mcause。

2. CSRU 根据异常号查询 medeleg 判断是否委托给 S-mode：若 `medeleg` 对应位为 1 且当前特权级不高于 S-mode，则陷入 S-mode（写 scause/sepc/stval，目标地址取 stvec）；否则陷入 M-mode（写 mcause/mepc/mtval，目标地址取 mtvec）。

3. 陷入时更新 mstatus：保存当前特权级到 MPP/SPP，保存当前中断使能位到 MPIE/SPIE，关闭中断使能（MIE/SIE 置零），切换到目标特权级。

4. CommitStage 生成 flush 信号清空流水线，redirect 指向 trap 入口地址。

由于异常处理严格在提交点进行，推测路径上的指令不会修改任何体系结构状态，因此异常对程序员始终表现为在触发异常的指令处精确发生。

7.4 中断机制

7.4.1 中断源与挂起检测

处理器支持两类中断源：

外部中断。外设（如 UART）产生的中断信号经 PLIC（Platform-Level Interrupt Controller）仲裁后，以 `ext_irq` 信号送入 CSRU，映射到 mip 的 SEIP 位（bit 9）。中断流为：外设 → PLIC → ext_irq → mip.SEIP。

定时器中断。CLINT（Core-Local Interruptor）维护全局计时器 mtime，CSRU 将 mtime 与 S-mode 定时器比较寄存器 stimecmp 比较，当 `mtime >= stimecmp` 时置位 mip 的 STIP 位（bit 5）。

软件中断。通过软件写 sip 寄存器的 SSIP 位（bit 1）触发。

7.4.2 中断使能与委托

中断的最终挂起状态由三个因素共同决定：mip（硬件挂起）、mie（使能掩码）与 mideleg（委托掩码）。CSRU 将中断分为两组计算：

- M-mode 中断：`m_interrupts = mip & mie & ~mideleg`（未委托的中断）
- S-mode 中断：`s_interrupts = mip & mie & mideleg`（已委托给 S-mode 的中断）

中断是否可响应还取决于当前特权级与全局中断使能位：M-mode 中断仅在 `priv < M` 或 `(priv == M && mstatus.MIE)` 时可响应；S-mode 中断仅在 `priv < S` 或 `(priv == S && mstatus.SIE)` 时可响应。

7.4.3 中断注入与优先级

中断的注入发生在 CommitStage：当 ROB 头部指令正常提交（无异常）且 CSRU 报告 `interrupt_pending` 为真时，CommitStage 先正常提交当前指令（更新 ArchRAT、释放 old_prd），随后立即触发中断 trap。中断的 epc 设置为当前提交指令的下一条 PC（retiring_dnpc），使中断返回后可从正确位置继续执行。

中断优先级遵循 RISC-V 规范：M-mode 中断优先于 S-mode 中断；同一模式内优先级为 MEI > MSI > MTI > SEI > SSI > STI。CSRU 通过 `MuxCase` 按优先级选择最高优先级的中断源，并输出对应的 cause 与 trap 目标地址。

7.4.4 MRET 与 SRET

mret 与 sret 指令在提交时触发特权级恢复：

- mret：从 mstatus.MPP 恢复特权级，将 mstatus.MPIE 恢复到 MIE，MPIE 置 1，MPP 清零，redirect 指向 mepc。
- sret：从 mstatus.SPP 恢复特权级，将 mstatus.SPIE 恢复到 SIE，SPIE 置 1，SPP 清零，redirect 指向 sepc。

7.5 Sv39 虚拟内存与内存管理单元

本项目实现了 RISC-V Sv39 虚拟内存方案。Sv39 将 64 位虚拟地址的低 39 位划分为三级页号（VPN[2:0]，每级 9 位）与 12 位页内偏移，通过三级页表将虚拟页号翻译为 44 位物理页号（PPN）。当 satp 寄存器的 MODE 字段为 8 时启用 Sv39，satp.PPN 指向根页表的物理基地址。

内存管理单元由 TLB（Translation Lookaside Buffer）与 PTW（Page Table Walker）两个模块组成，IFU 与 LSU 各自实例化一套独立的 iTLB+PTW 和 dTLB+PTW，分别服务于取指地址翻译与访存地址翻译。两套 MMU 在结构上完全相同，仅在权限检查的具体位（IFU 检查 PTE.X，LSU 检查 PTE.R/PTE.W）上有所区别。

7.5.1 TLB 设计

TLB 采用 16 项全相联结构，以 VPN（27 位）作为 tag 进行匹配查询。查询时对所有表项并行比较，通过优先级编码器选中命中项，返回对应的 PPN 与页表标志位（flags）。sfence.vma 指令执行时，通过 flush 信号一次性失效全部表项。

TLB 的替换策略采用位矩阵 LRU（Bit-Matrix LRU）算法。该算法使用一个 N×N 的位矩阵 M 记录表项间的访问先后关系：M[i][j] = 1 表示表项 i 比表项 j 更近被访问。当访问表项 k 时，将第 k 行全部置 1、第 k 列全部置 0，表示 k 成为最近访问项。需要淘汰时，选择行向量全为 0 的表项（即比所有其他表项都更早被访问的项）作为替换目标。算法伪代码如下：

```
# 访问表项 k 时更新矩阵
def access(M, k, N):
    for j in range(N):
        M[k][j] = 1   # 第 k 行全部置 1
    for i in range(N):
        M[i][k] = 0   # 第 k 列全部置 0

# 选择 LRU 表项（行全零）
def find_lru(M, N):
    for i in range(N):
        if all(M[i][j] == 0 for j in range(N)):
            return i
```

代码 7-1　位矩阵 LRU 替换算法伪代码

位矩阵 LRU 的硬件实现仅需 N×(N-1)/2 个触发器（矩阵对角线恒为 0 且 M[i][j] = !M[j][i]，实际只需存储上三角），访问更新与 LRU 查询均可在单周期内完成，适合全相联结构的小容量 TLB。

7.5.2 PTW 设计

PTW 负责在 TLB 缺失时遍历内存中的三级页表。PTW 采用九状态有限状态机实现：

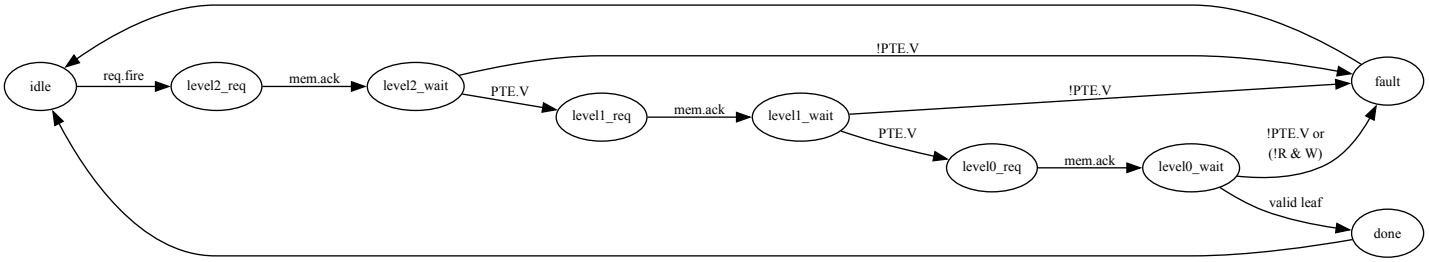

图 7-2　PTW 页表遍历状态机

如 图 7-2 所示，PTW 从 idle 状态接受请求后，依次访问第 2 级、第 1 级与第 0 级页表项。每一级的访问地址由上一级 PTE 中的 PPN 拼接当前级的 VPN 索引与 3 位字节偏移构成（每个 PTE 为 8 字节）。每级读取 PTE 后检查有效位（PTE.V）：若无效则进入 fault 状态报告页错误；若有效则继续下一级。第 0 级还额外检查权限合法性（不允许 !R && W 的非法编码）。遍历成功后进入 done 状态，输出叶子 PTE 的 PPN 与 flags 供 TLB refill 使用。

7.5.3　IFU 中的地址翻译

IFU 的取指状态机中集成了 iTLB 查询与 PTW 调用。在 tlb_check 状态：若 Sv39 未启用则直接使用虚拟地址作为物理地址；若 iTLB 命中则检查 PTE.X（执行权限），权限合法则使用翻译后的物理地址进入 ICache 访问，权限不合法则标记 instruction page fault（mcause = 12）；若 iTLB 缺失则启动 PTW 遍历页表，PTW 完成后 refill iTLB 并回到 tlb_check 重新查询。这一“乐观路径先查 TLB、悲观路径回退到 PTW”的设计使得 TLB 命中时无额外延迟，TLB 缺失的代价由 PTW 的三次内存访问决定。

7.5.4　LSU 中的地址翻译

LSU 的访存状态机采用与 IFU 相同的 TLB 优先策略：

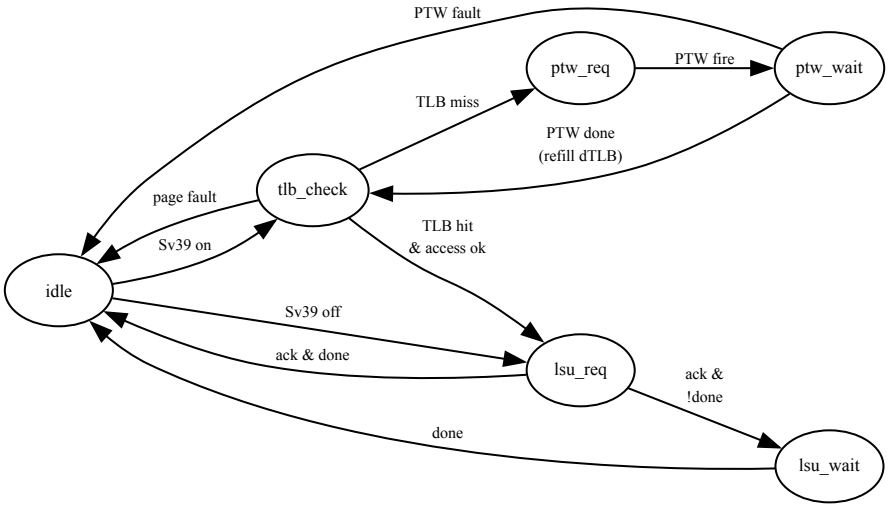

图 7-3　LSU 访存状态机（含地址翻译）

如图 7-3 所示，ROB 发起访存请求后，LSU 首先判断 Sv39 是否启用。若未启用则直接使用虚拟地址进入 lsu_req 访问 DCache 或外设端口；若启用则进入 tlb_check 查询 dTLB。dTLB 命中时检查权限：Load 指令检查 PTE.R，Store 指令检查 PTE.W，权限不合法则报告 load page fault（mcause = 13）或 store page fault（mcause = 15）并回到 idle。权限合法则将翻译后的物理地址锁存到 `pa` 寄存器，进入 lsu_req。dTLB 缺失时启动 PTW，PTW 完成后 refill dTLB 并回到 tlb_check 重新查询。

lsu_req 状态根据地址映射判断访问目标：若地址属于 MMIO 区域则通过外设端口（perip）访问，否则通过 DCache 端口访问。两条路径共享相同的握手逻辑：单周期完成时直接返回 idle，否则进入 lsu_wait 等待数据就绪。Load 指令的返回数据根据访问宽度（byte/half/word/double）与符号扩展标志进行对齐提取与扩展后写回 PRF。

7.6　本章小结

本章介绍了 RISC-V M/S/U 三级特权架构、CSR 寄存器文件、精确异常、中断机制与 Sv39 虚拟内存的设计与实现。精确异常通过 ROB 顺序提交保证，异常仅在提交点生效；中断通过 CSRU 的挂起检测与 CommitStage 的注入机制实现，支持外部中断（PLIC）、定时器中断（CLINT/stimecmp）与软件中断三类来源，并通过 medeleg/mideleg 实现异常与中断的 M/S 两级委托。Sv39 虚拟内存由 TLB 与 PTW 两个模块实现，IFU 与 LSU 各自拥有独立的 iTLB/dTLB 与 PTW，采用“乐观路径先查 TLB、悲观路径回退到 PTW”的状态机设计，在 TLB 命中时无额外延迟。

第 8 章　缓存设计

8.1　类 SRAM 总线协议

处理器核心（IFU、LSU）与缓存之间的交互采用自定义的类 SRAM（SRAM-like）总线协议，而非直接使用 AXI4[31] 等复杂片上总线。这一设计的出发点是降低处理器核心侧的接口复杂度：AXI4 协议包含 5 个独立通道（AR、AW、W、R、B）与多种突发传输模式，对于处理器核心每次仅发起单拍读写的场景过于冗余；而类 SRAM 协议将读写统一到一组信号上，核心侧仅需驱动一个请求端口即可完成访存操作。

```
class SRAMBundle(val params: SRAMBundleParameters) extends Bundle {
  val req   = Output(Bool())      // 请求有效
  val wen   = Output(Bool())      // 写使能（0=读，1=写）
  val size  = Output(UInt(...))   // 访问宽度
  val addr  = Output(UInt(...))   // 地址
  val wstrb = Output(UInt(...))   // 写字节掩码
  val wdata = Output(UInt(...))   // 写数据
  val ack   = Input(Bool())       // 请求被接受
  val done  = Input(Bool())       // 数据就绪 / 写完成
  val rdata = Input(UInt(...))    // 读数据
}
```

代码 8-1　类 SRAM 总线接口定义

协议采用 req → ack → done 三阶段握手。图 8-1 与 图 8-2 分别展示了一次读事务与一次写事务的时序波形。

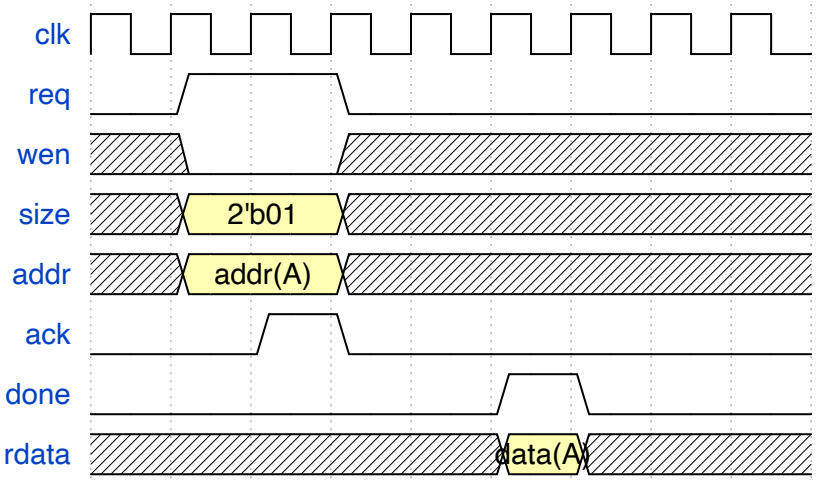

图 8-1　类 SRAM 总线上的一次读事务

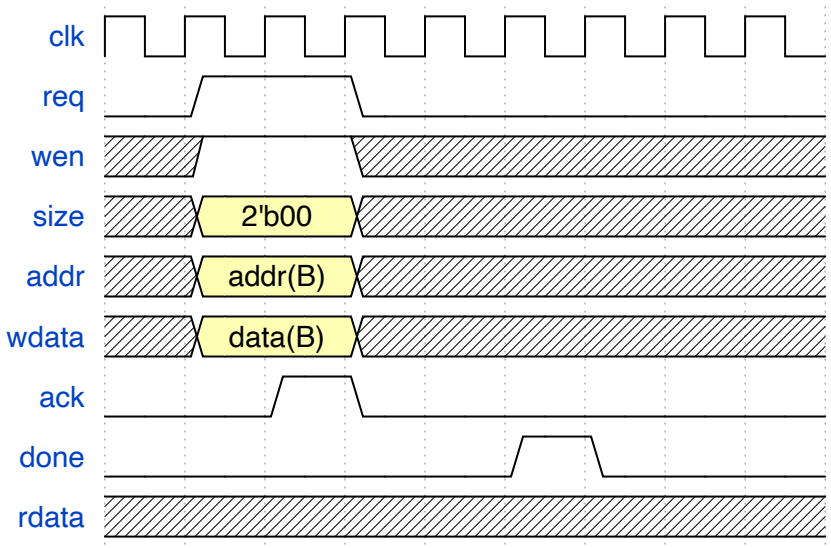

图 8-2　类 SRAM 总线上的一次写事务

如 图 8-1 所示，主设备发起读请求时拉高 req 并驱动 addr、wen=0、size 等信号，req 持续保持高电平直到从设备拉高 ack，此时两者同时拉低，完成一次请求握手（req-ack）。随后从设备在数据就绪时拉高 done 并在 rdata 上输出有效数据，完成数据响应。写事务（图 8-2）的握手流程与读事务相同，区别在于 wen=1 且主设备额外驱动 wdata 与 wstrb。

该协议的核心设计在于将 req-ack 请求握手与 done 数据响应分离为两个独立阶段。这一分离带来两方面优势：

1. 避免复杂协议。处理器核心与 L1 Cache 之间无需使用 AXI4 的五通道握手协议，仅需驱动 req 和检测 ack/done 两对信号，显著降低了核心侧的接口复杂度与控制逻辑面积。

2. 保持流水能力。对于读操作，地址阶段（req-ack）与数据阶段（done）天然分离：ack 返回后地址已被缓存锁存，核心状态机可提前进入等待数据状态。当缓存命中时，ack 与 done 可在同一周期返回，实现单周期完成；当缓存缺失时，ack 先返回表示请求已接受，done 在缓存行填充完成后延迟返回。核心侧状态机无需区分命中与缺失，统一以“等待 done”收束。

8.1.1　Diplomacy 参数协商

为使类 SRAM 总线能够融入 Rocket Chip[32] 的 Diplomacy 参数协商框架，本项目为其实现了完整的节点体系。Diplomacy 框架的核心思想是：总线参数（地址宽度、数据宽度、地址映射等）不在模块实例化时硬编码，而是由连接图中的上下游节点在编译期自动协商确定。

具体实现包括：`SRAMMasterPortParameters` 与 `SRAMSlavePortParameters` 描述主从端口的参数约束（地址范围、数据位宽等）；`SRAMEdgeParameters` 在连接建立时由框架自动生成，包含协商后的 `SRAMBundleParameters`；`SRAMImp` 实现 `SimpleNodeImp` 接口，定义参数协商规则与硬件 Bundle 生成逻辑。在此基础上派

生出 `SRAMMasterNode`、`SRAMSlaveNode`、`SRAMNexusNode` 与 `SRAMIdentityNode`，分别用于源节点、汇节点、多端口汇聚与透传场景。

8.1.2 SRAMToAXI4 桥接与缓存的双面接口

类 SRAM 协议服务于处理器核心侧，而 SoC 互连采用 AXI4 协议，因此需要在两者之间进行协议转换。本项目实现了通用的 `SRAMToAXI4` 桥接模块，用于不经过缓存的直通路径（如外设端口）。桥接模块内部采用五状态 FSM（idle → readAddr → readData / write → writeResp），将 SRAM 的单拍读写映射为 AXI4 的单拍突发传输（len=0）。该模块基于 Diplomacy 的 `MixedAdapterNode` 实现，上游为 SRAM 节点、下游为 AXI4 节点，参数在连接时自动转换。

ICache 与 DCache 本身也充当协议桥接的角色：面向处理器核心暴露类 SRAM 接口，接收单拍读写请求；面向 SoC 互连则通过 AXI4 接口与下级存储交互。缓存缺失时，缓存控制器通过 AXI4 Burst 传输（len > 0）从主存读取整条缓存行完成 refill；写回脏行时同样通过 AXI4 Burst 将整条缓存行写回主存。这一设计使处理器核心始终面对简洁的类 SRAM 接口，缓存内部负责单拍请求与突发传输之间的转换。

8.2 PLRU 替换策略

ICache 与 DCache 均为 4 路组相联结构（64 组，每组 4 路，缓存行 64 字节）。替换策略采用伪 LRU（Pseudo-LRU, PLRU）算法[33]，以较低的硬件开销近似 LRU 行为。

PLRU 使用一棵 $n-1$ 个节点的二叉树（4 路对应 3 个节点）记录访问方向。每个节点存储 1 位标志，指向“最近较少使用”的子树方向。如 图 8-3 所示，树的叶子节点对应 4 条缓存路，内部节点的标志位在每次访问时更新为指向另一侧，使得沿标志位路径走到的叶子即为待替换的路。

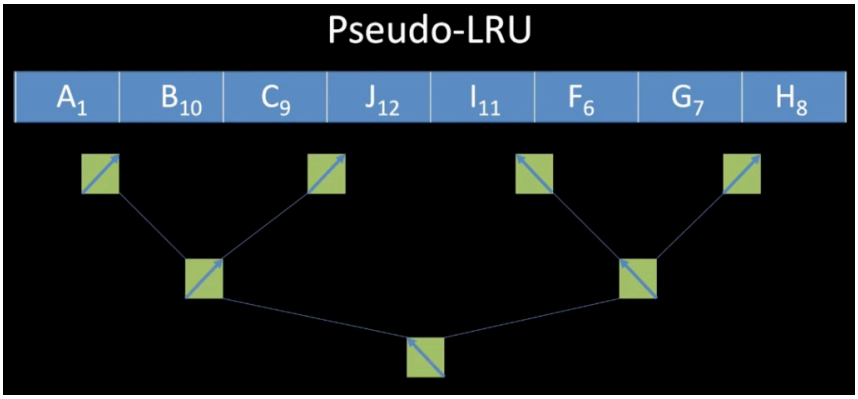

图 8-3　4 路 Pseudo-LRU 二叉树结构示意

```python
def plru_access(tree: list[int], way: int):
    """访问 way 后更新 PLRU 树节点，使其指向远离被访问路的方向"""
    tree[0] = 1 if way in (0, 1) else 0  # 根节点
    tree[1 + way // 2] = 1 if way in (0, 2) else 0  # 子节点
```

代码 8-2　PLRU 访问更新

```python
def plru_victim(tree: list[int]) -> int:
    """沿树节点标志位从根到叶，返回待替换的路号"""
    if tree[0] == 0:  # go left
        return 0 if tree[1] == 0 else 1
    else:             # go right
        return 2 if tree[2] == 0 else 3
```

代码 8-3　PLRU 替换选择

如 代码 8-2 所示，每次缓存命中时根据命中的路号更新树节点标志，使其指向远离被访问路的方向。如 代码 8-3 所示，需要替换时沿树节点标志位从根到叶遍历，得到的叶子即为牺牲路。该算法仅需 3 位状态与简单的组合逻辑即可实现 4 路近似 LRU 替换，硬件开销远低于完整 LRU 所需的访问序记录。

8.3　ICache

ICache 采用 VIPT（Virtually Indexed, Physically Tagged）结构，64 组 x 4 路 x 64 字节缓存行，总容量 16 KB。面向处理器核心提供类 SRAM 只读接口，面向 SoC 互连通过 AXI4 接口进行 refill。ICache 为只读缓存，不涉及写操作与脏行管理。

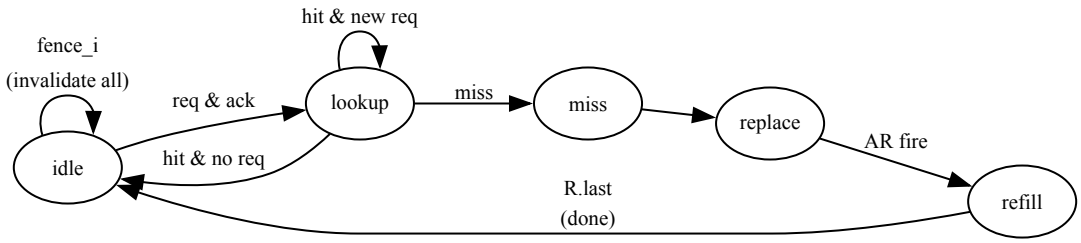

图 8-4　ICache 主状态机

如 图 8-4 所示，ICache 采用五状态 FSM：

idle：等待请求。收到 fence.i 信号时一次性失效全部缓存行。收到 req 时锁存地址（拆分为 tag、index、beat 偏移），返回 ack 并进入 lookup。

lookup：用锁存的 tag 与 index 查询 tag RAM，4 路并行比较。若命中，直接从 data bank 读出目标 beat 的数据，返回 done 与 rdata。命中时若有新请求可直接锁存并停留在 lookup（lookup -> lookup 快速路径），实现连续命中的单周期吞吐。若缺失，记录 PLRU 选出的替换路号，进入 miss。

miss -> replace：miss 为单周期过渡状态。replace 状态向 AXI4 AR 通道发起突发读请求（len=7，8 拍 x 8 字节 = 64 字节），AR 握手成功后进入 refill。

refill：逐拍接收 AXI4 R 通道返回的数据并写入 data bank 对应路的各 beat。当接收到目标 beat 时额外锁存其数据。收到 R.last 后更新 tag RAM（写入新 tag、置 valid）、推进 PLRU 替换指针，返回 done 与 rdata，回到 idle。

8.4 DCache

DCache 同样采用 VIPT 结构，64 组 x 4 路 x 64 字节缓存行，总容量 16 KB。与 ICache 的关键区别在于 DCache 需要处理写操作，采用写回（write-back）策略，并引入 Write Buffer 与脏行写回机制。

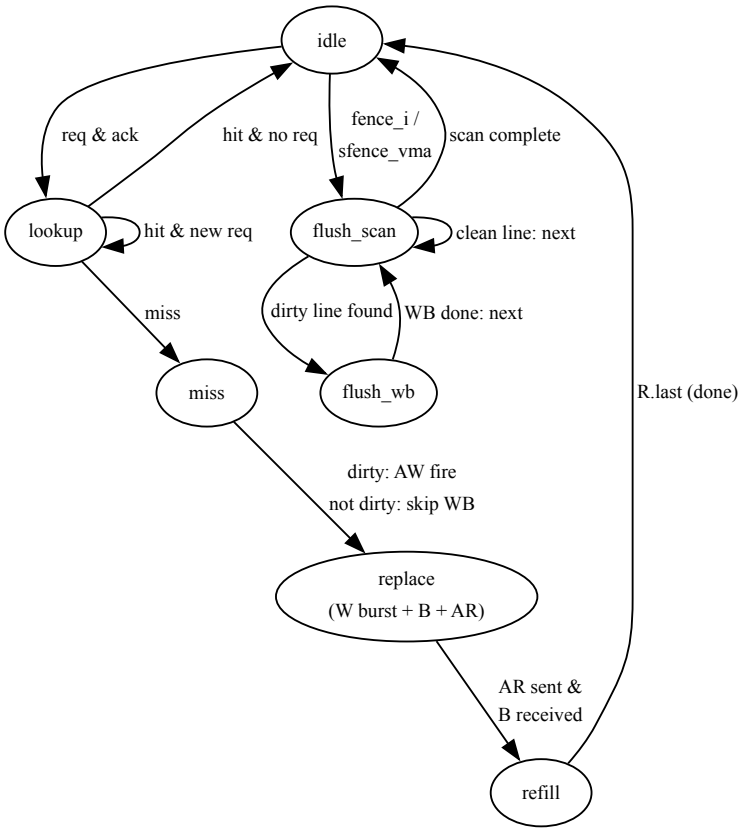

图 8-5　DCache 主状态机

如 图 8-5 所示，DCache 的主状态机包含七个状态：

1. idle：等待请求。收到 fence.i 或 sfence.vma 时进入 flush 流程；正常请求时锁存地址与写控制信号（wen、wdata、wstrb），进入 lookup。
2. lookup：4 路并行比较。命中时：若为读操作直接返回数据；若为写操作触发 Write Buffer——将写入目标（index、way、beat、wstrb、wdata）送入单项 Write Buffer，在下一周期由 Write Buffer FSM 完成按字节掩码合并写入 data bank 并置脏位。Write Buffer 的引入使写命中不阻塞主 FSM，lookup 可立即接受下一请求。Write

Buffer 还负责检测读写冲突：若新读请求与正在执行的 Write Buffer 写入地址重叠，则暂停接受请求直到 Write Buffer 完成。

3. miss：PLRU 选出替换路后，检查牺牲行是否为脏行。若脏则先通过 AXI4 AW 通道发起写回地址；若干净则跳过写回阶段。
4. replace：该状态并行处理三个子阶段——（1）通过 AXI4 W 通道逐拍写回脏行数据；（2）等待 AXI4 B 通道确认写回完成；（3）通过 AXI4 AR 通道发起 refill 读请求。当 AR 发送完毕且 B 确认收到后进入 refill。
5. refill：逐拍接收 R 通道数据写入 data bank。若原始请求为写操作，在目标 beat 位置将 refill 数据与写数据按字节掩码合并后写入。R.last 后更新 tag RAM（新 tag、valid、dirty 标志）、推进 PLRU 指针，返回 done 回到 idle。
6. flush_scan -> flush_wb：fence.i 或 sfence.vma 触发全缓存刷写。flush_scan 逐组逐路扫描，遇到脏行则进入 flush_wb 通过 AXI4 写回该行，写回完成后清除脏位并继续扫描，直到所有组路遍历完毕回到 idle。

8.5 指令-数据缓存一致性与 fence.i

在冯·诺依曼架构中，指令与数据共享同一地址空间；然而本处理器采用分离式 L1 缓存（ICache 与 DCache），二者之间不存在硬件一致性协议。这意味着当程序在运行时修改自身代码——即“自修改代码”（Self-Modifying Code）场景——时，ICache 与 DCache 之间会出现一致性问题。

一个典型的实例是 Bootloader 的快速启动路径：系统上电后，处理器从 XIP（Execute-In-Place）模式的 Flash 开始执行，Bootloader 将 Flash 中的应用程序映像搬运到 SDRAM 以获得更高的访问速度，随后跳转至 SDRAM 入口地址继续执行。其核心指令序列如下：

步骤	指令
1	`sw inst, 0(sdram_addr)` —— 将指令写入 SDRAM（经 DCache）
2	`j sdram_addr` —— 跳转至 SDRAM 入口执行

问题在于：步骤 1 的 `sw` 指令将数据写入 DCache（写回策略下，脏行尚未写回 SDRAM）；步骤 2 的跳转触发 IFU 从 `sdram_addr` 取指，而 IFU 只能访问 ICache 与下级存储，无法看到 DCache 中的脏数据。此时 ICache 未命中，从 SDRAM 读取到的仍是旧数据（全零或无效内容），导致处理器执行错误指令。

RISC-V 指令集架构通过 `fence.i` 指令显式解决这一问题。正确的指令序列应为：

步骤	操作
1	`sw inst, 0(sdram_addr)` —— 将指令写入 SDRAM（经 DCache）
2	`fence.i` —— 同步指令流与数据流
3	`j sdram_addr` —— 跳转至 SDRAM 入口执行

在本处理器的实现中，`fence.i` 的执行分为两个阶段：

1. DCache 刷写：DCache 进入 flush 流程（参见 图 8-5 中 flush_scan → flush_wb 路径），逐组逐路扫描所有缓存行，将脏行通过 AXI4 写通道写回下级存储，确保所有已修改数据对外可见。
2. ICache 失效：DCache 刷写完成后，ICache 将全部缓存行的 valid 位清零，迫使后续取指请求从下级存储重新加载。

经过 `fence.i` 的两阶段处理，步骤 1 写入 DCache 的指令数据已安全落入 SDRAM，而 ICache 的全部内容已失效。当步骤 3 跳转到 `sdram_addr` 时，ICache 必然未命中，从 SDRAM 重新加载到正确的指令内容，程序得以正确执行。

需要指出的是，`fence.i` 是一条代价较高的指令：它要求刷写整个 DCache 并失效整个 ICache，开销与缓存容量成正比。RISC-V 规范将其定义为“Zifencei”扩展而非基础指令集的一部分，正是考虑到在多核系统中维护指令-数据一致性有更高效的硬件机制（如 RISC-V CMO 扩展）。在本处理器的单核场景下，`fence.i` 足以保证正确性，且实现简洁。

8.6 本章小结

本章围绕缓存子系统展开，首先介绍了面向处理器核心的类 SRAM 总线协议，其 req-ack-done 三信号握手机制在保持接口简洁的同时支持读地址与读数据的流水分离；随后描述了基于 Diplomacy 框架的参数协商与 SRAMToAXI4 总线桥的设计，使 ICache 与 DCache 对内呈现 SRAM 接口、对外通过 AXI4 Burst 完成 refill 与 writeback。在替换策略方面，采用 Pseudo-LRU 算法以 3 位状态实现 4 路近似最近最少使用替换，兼顾硬件面积与替换效果。ICache 采用五状态有限状态机实现只读缓存逻辑，DCache 则通过七状态有限状态机支持写回策略、Write Buffer 优化及全缓存刷写。最后讨论了分离式 ICache 与 DCache 架构下的指令-数据一致性问题，说明了 `fence.i` 指令通过 DCache 刷写加 ICache 失效两阶段操作保证自修改代码的正确执行。

第 9 章　仿真平台

处理器设计的正确性高度依赖仿真验证。本章介绍基于 Verilator 与 C++20 构建的仿真平台，涵盖简易调试器、执行追踪、差分测试、快照回放及外设可视化等功能，为处理器开发全流程提供调试与验证支撑。

9.1　仿真平台整体架构

仿真平台以 C++20 编写，遵循 Google C++ Style Guide 编码规范，使用 CMake 构建系统管理依赖。整体架构如 图 9-1 所示，由以下核心组件构成：

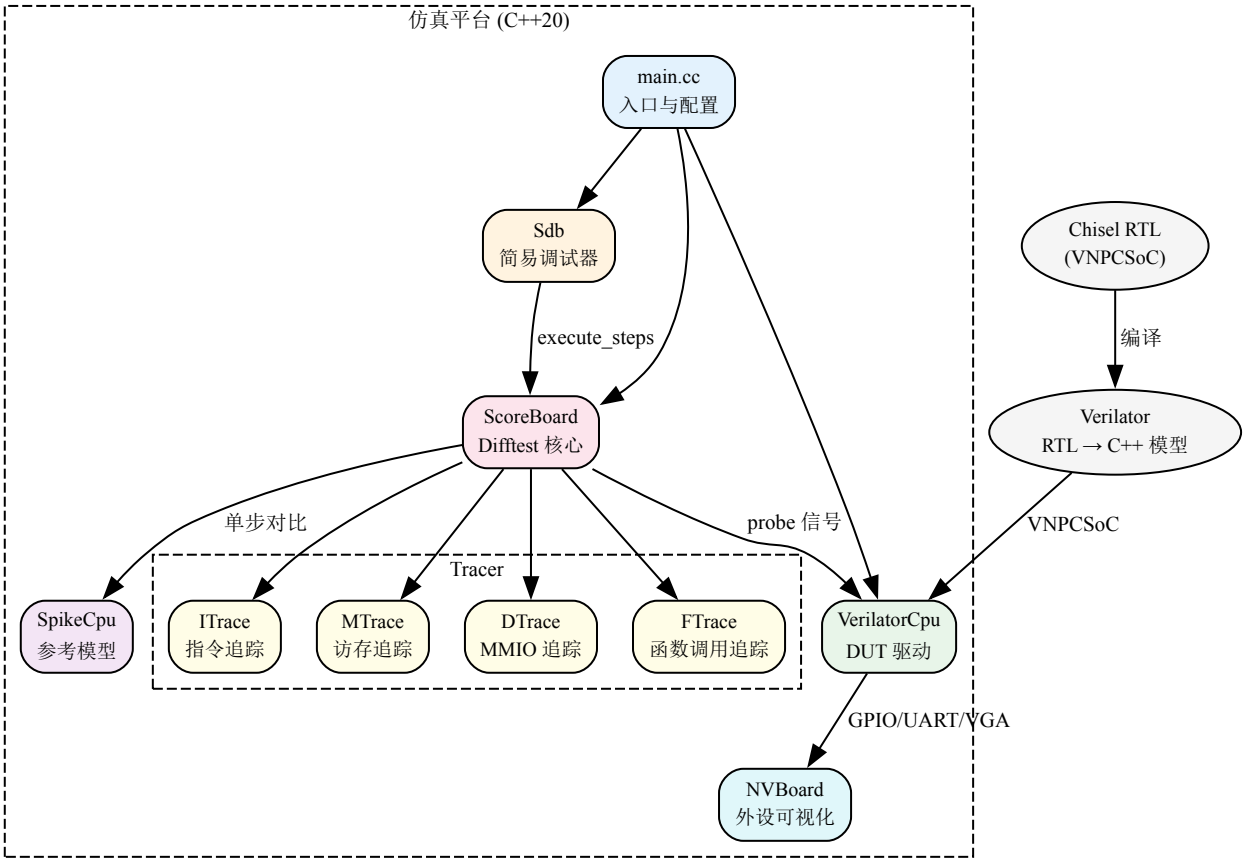

图 9-1　仿真平台整体架构

Verilator 将 Chisel 生成的 Verilog 网表编译为 C++ 仿真模型 `VNPCSoC`。`VerilatorCpu` 封装该模型并驱动时钟信号，通过 Chisel 的 probe 机制从 RTL 中导出 PC、通用寄存器、CSR、提交有效信号等观测点，供上层调试与比对使用。`ScoreBoard` 是差分测试与追踪记录的核心枢纽：每条指令提交后，它同步驱动参考模型 `SpikeCpu` 执行同一条指令，比较两侧架构状态，并将执行信息写入各类 Tracer 的环形缓冲区。`Sdb`（Simple Debugger）提供交互式调试界面，支持单步执行、断点、监视点与表达式求值。命令行标志 `--batch` 可使仿真在无人值守模式下全速运行，`--wave` 可按需开启波形记录。

9.2 简易调试器

Sdb 提供类 GDB 的交互式命令行界面，基于 GNU Readline 实现命令补全与历史记录。支持的调试命令包括：

命令	缩写	功能
`continue`	`c`	继续执行直到断点或程序结束
`step`	`si`	单步执行 N 条指令（默认 1）
`info r`		显示所有通用寄存器与 PC
`info w`		列出所有监视点
`print`	`p`	对表达式求值并输出
`examine`	`x`	查看指定地址处的内存内容
`watch`	`w`	添加监视点，表达式值变化时暂停
`break`	`b`	在指定地址设置断点
`delete`	`d`	删除指定监视点

9.2.1 表达式求值引擎

Sdb 内置了一个完整的表达式求值引擎，使用 Flex 进行词法分析、Bison 进行语法分析。该引擎支持以下语法特性：

- 字面量：十进制整数、十六进制整数（`0x` 前缀）。
- 寄存器引用：`$pc`、`$ra`、`$sp` 等，通过 `AbstractCpu::value()` 接口读取当前 CPU 状态。
- 算术运算：`+`、`-`、`*`、`/`。
- 比较运算：`==`、`!=`、`<`、`<=`、`>`、`>=`。
- 逻辑运算：`&&`、`||`。
- 一元运算：负号 `-`、解引用 `*addr`（读取指定地址的内存值）。

表达式求值贯穿调试器的多个功能：`print` 命令直接求值输出，`examine` 命令使用表达式计算起始地址，`watch` 命令在每步执行后重新求值以检测变化。

9.3 执行追踪

仿真平台实现了四类追踪器（Tracer），均基于泛型环形缓冲区 `RingBuf<T>` 存储最近的追踪记录。环形缓冲区采用固定容量、覆写最旧记录的策略，在常数内存开销下保留最近的执行历史，供错误发生时回溯分析。

9.3.1 ITrace：指令追踪

ITrace 记录每条提交指令的 PC、机器码与反汇编文本。反汇编由 Spike 的内置反汇编器生成，保证与参考模型的指令解读一致。当 difftest 失败时，ITrace 的输出可直接定位出错指令及其上下文。

9.3.2 MTrace：访存追踪

MTrace 记录所有非 MMIO 的 load/store 操作，包括 PC、访存方向（读/写）、物理地址、数据值与访问宽度。对于 load 指令，数据值取自 DUT 的内存模型；对于 store 指令，数据值取自参考模型的寄存器。MTrace 对缓存一致性问题与访存相关 bug 的定位尤为有效。

9.3.3 DTrace：设备追踪

DTrace 专门记录 MMIO 区域的访问，格式与 MTrace 相同但独立存储。MMIO 访问具有副作用（如串口发送、定时器读取），其正确性无法通过简单的内存比对验证，因此单独追踪便于开发者审查外设交互序列。

9.3.4 FTrace：函数调用追踪

FTrace 从 ELF 文件的符号表中加载函数名与地址映射，在运行时根据指令助记符识别函数调用（`jal` / `jalr` 且 `rd=ra`）与返回（`ret`），维护调用深度计数器，生成带缩进的调用链记录。FTrace 对操作系统启动流程的调试尤为有用：当程序陷入死循环或异常跳转时，调用链能清晰展示控制流的偏离点。

9.4 差分测试

差分测试（Differential Testing，简称 Difftest）[20] 是本仿真平台最核心的验证手段。

9.4.1 理论基础：处理器的双层状态机模型

从抽象层次看，处理器本质上是一个状态机，其状态可分为两个层次：

- ISA 状态（Instruction Set Architecture State）：程序员可见的架构状态，包括程序计数器 PC、32 个通用寄存器、CSR 寄存器以及内存内容。ISA 规范严格定义了每条指令对这些状态的转移规则，任何符合规范的实现——无论是单周期、多周期还是乱序流水线——在执行同一条指令后都必须产生相同的 ISA 状态。
- 物理状态（Physical/Microarchitectural State）：硬件实现的微架构状态，包括流水线寄存器、重排序缓冲区、缓存行、分支预测器历史等。这些状态对程序员不可见，不同微架构实现之间存在本质差异。

程序员编写软件时只需关心 ISA 状态——这正是 ISA 作为硬件与软件之间契约的核心含义。即便存在诸如 `fence.i` 这样涉及微架构行为的指令，其语义仍然

是在 ISA 层面定义的（“使后续取指看到此前的数据写入”），而非暴露具体的缓存实现细节。

基于这一认识，差分测试的思路自然浮现：如果存在一个公认正确的 ISA 状态机作为参考，让待验证的处理器（DUT）与之逐条指令执行同一程序，每步比较 ISA 状态，若始终一致，即可断言 DUT 的实现在 ISA 语义上是正确的。RISC-V 基金会维护的官方指令集模拟器 Spike[21] 正是这样一个标准的 ISA 参考状态机：它严格按照 RISC-V 规范实现每条指令的状态转移，不涉及任何微架构细节，是差分测试的理想参考模型。

9.4.2 参考模型集成

本平台以静态链接方式集成 Spike（`spike::riscv` 库），而非通过动态库加载，使仿真平台的部署更为简洁。`SpikeCpu` 类封装了 Spike 的 `sim_t`、`processor_t` 与 `state_t`，并按照 SoC 的实际地址映射（Flash 位于 `0x30000000`、SDRAM 等）初始化 Spike 的内存布局，确保两侧的初始 ISA 状态完全一致。

9.4.3 比对流程

差分测试的执行流程由 `ScoreBoard::scoreboard()` 驱动，每条指令提交后执行以下步骤：

1. DUT 侧：`VerilatorCpu::step()` 驱动时钟直到 `probe_valid` 信号有效，表示一条指令已提交。通过 probe 信号读取 DUT 的 PC、下一 PC（`dnpc`）、机器码、通用寄存器与 CSR——这些正是 ISA 状态的完整快照。
2. 参考模型侧：调用 `SpikeCpu::step()` 执行同一条指令，Spike 按照 ISA 规范更新其内部状态。
3. ISA 状态比对：`check_regs()` 逐一比较 DUT 与 Spike 的下一 PC、32 个通用寄存器（`x1`～`x31`）以及关键 CSR（`mtvec`、`mepc`、`mcause`、`mtval`），任一不匹配即判定 DUT 的 ISA 行为偏离规范。

9.4.4 MMIO 特殊处理

MMIO 访问具有副作用且地址空间在 DUT 与 Spike 之间不一定对等，因此需要特殊处理。当 DUT 的 `is_mmio` 信号有效时，ScoreBoard 不调用 `golden_.step()`，而是：

- 对 MMIO load：从 DUT 的目的寄存器读取实际值，写入 Spike 对应寄存器，使两侧对齐。
- 对 MMIO store：记录到 DTrace，不修改 Spike 内存。
- 最后将 Spike 的 PC 强制设置为 DUT 的 `dnpc`，跳过 Spike 对该指令的执行。

这种“DUT 主导、参考模型跟随”的策略确保了 MMIO 访问不会导致两侧状态分叉。

9.4.5 失败诊断

当 difftest 检测到不一致时，`ScoreBoard::dump_traces()` 按序输出四类 Tracer 的环形缓冲区内容，并以表格形式并排展示 DUT 与 Spike 的寄存器与 CSR 状态，

用 `<--- MISMATCH` 标记不一致项。开发者可据此快速定位出错指令及其影响的架构状态。

9.5 LightSSS 快照回放

在长时间仿真过程中，某些 bug（如总线死锁、缓存状态机卡死）可能在数百万周期后才触发，此时重新从头仿真并开启波形记录的开销极为高昂。LightSSS（Lightweight Simulation Snapshot）利用操作系统的 `fork()` 系统调用实现轻量级快照功能：

1. 周期性快照：仿真每执行一定数量的指令后，调用 `fork()` 创建子进程。由于 `fork()` 采用写时复制（Copy-on-Write）语义，快照的创建几乎零开销——父子进程共享相同的物理内存页，仅在后续写入时才触发页复制。
2. 异常检测：当仿真检测到异常状态（如 `VerilatorCpu::step()` 超过最大周期数仍未收到 `probe_valid`，表明总线可能死锁），触发快照回放机制。
3. 死亡回放：系统回退到最近一个 checkpoint 对应的子进程，在该快照上重新执行，但此次开启波形记录（`--wave`）与全部 Tracer，以最小的时间窗口捕获 bug 触发前后的完整信息。

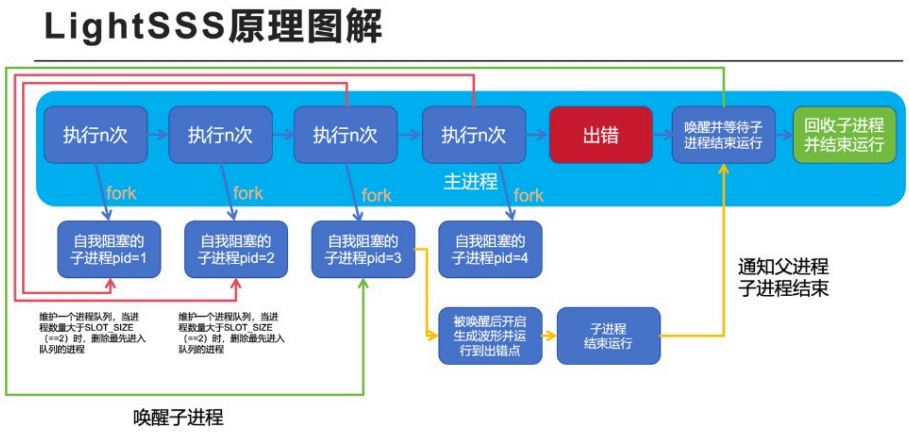

图 9-2　LightSSS 快照回放原理示意

如图 9-2 所示，主进程每执行 N 条指令后调用 `fork()` 创建一个自我阻塞的子进程作为快照，并维护一个固定大小的进程队列（SLOT_SIZE = 2），当快照数量超出上限时淘汰最早的快照。当主进程检测到仿真出错时，唤醒最近的快照子进程，该子进程开启波形记录并重新运行到出错点，完成“死亡回放”后通知父进程回收资源。这一机制的关键优势在于：正常仿真时无需开启波形记录（可提升 5~10 倍仿真速度），仅在 bug 触发时才回放并记录，大幅缩小了仿真日志与波形文件的体积。

9.6 NVBoard 外设可视化

仿真平台集成了 NVBoard(NJU Virtual Board) 虚拟开发板，通过将 RTL 顶层的 GPIO、七段数码管、PS2 键盘、VGA 显示与 UART 串口等外设信号绑定到 NVBoard 的虚拟引脚，在仿真过程中实时渲染外设状态。通过命令行标志 `--nvboard` 启用。

对于 UART 输出，仿真平台实现了 `UartTxDecoder` 模块，在仿真时钟域内解码 8N1 串口帧：通过追踪 TX 引脚电平变化的最小脉冲宽度自动检测波特率，在每个数据位的中心采样，完成字节解码后送入 NVBoard 的 UART 终端显示。

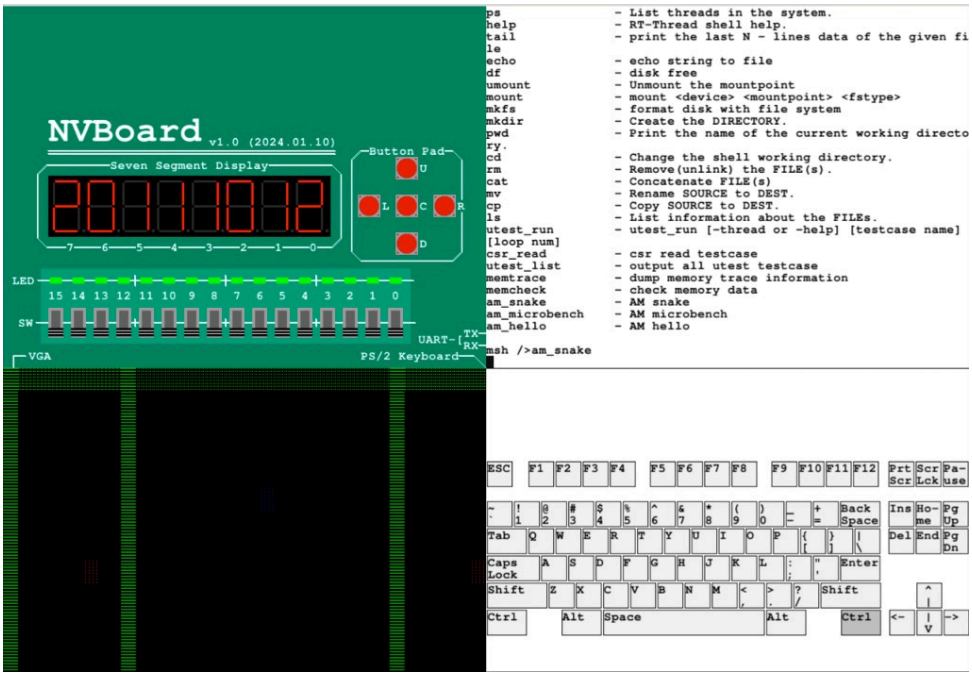

图 9-3　NVBoard 虚拟开发板运行 RT-Thread 操作系统上的贪吃蛇程序

图 9-3 展示了 NVBoard 虚拟开发板的运行截图，整个程序运行在 RT-Thread[24] 实时操作系统之上。左上角为 GPIO 区域，七段数码管显示学号后 8 位；右上角为 UART 串口终端与 PS/2 键盘回显，可见 RT-Thread 的 msh（Mini Shell）命令行界面及其内置命令列表；左下角为 VGA 显示区域，正在运行贪吃蛇程序；右下角为 PS/2 虚拟键盘，用于向处理器发送按键扫描码。各外设在仿真过程中实时更新，为软硬件协同调试提供直观的可视化反馈。

9.7　本章小结

本章介绍了基于 Verilator 与 C++20 构建的仿真验证平台。该平台以 Google C++ Style Guide 为编码规范，围绕“调试—追踪—比对—回放—可视化”五个维度构建了完整的验证工具链：Sdb 简易调试器提供交互式单步执行与断点调试，内置基于 Flex/Bison 的表达式求值引擎；四类 Tracer（ITrace、MTrace、DTrace、FTrace）基于环形缓冲区记录指令、访存、MMIO 与函数调用历史；差分测试以 Spike 为参考模型，逐条指令比对架构状态，MMIO 访问采用 DUT 主导策略保持两侧同步；LightSSS 利用 `fork()` 写时复制实现轻量级快照，在 bug 触发时自动回放并开启波形记录；NVBoard 集成提供 GPIO、UART、VGA 等外设的实时可视化。上述工具的协同使用，使得从功能验证到性能调优的全流程均有据可查、有迹可循。

结　论

本项目围绕“RISC-V 指令集乱序处理器设计”这一课题，完成了从核心微架构设计到系统级集成与验证的全流程工作。主要成果与结论如下：

1. SoC 系统设计与集成。设计并实现了基于 AXI4/APB 分层互连的 SoC 平台，通过一级 Crossbar 服务高带宽主存路径、二级 Crossbar 连接片上存储与外设桥接，形成了处理器核心、总线、存储控制器与外设协同工作的完整系统环境。该平台为乱序处理器运行操作系统提供了稳定的硬件基础。
2. 关键存储控制器 IP 核实现。使用 Chisel 实现了 Flash 控制器（支持 SPI 命令访问与 XIP 直接取指执行）、PSRAM QSPI Master 控制器（支持 QSPI/QPI 模式切换）与 SDRAM 控制器（支持流水化访问与低位交叉字扩展），三者均配有行为级仿真模型并通过协议级验证，具备对接物理芯片的能力。
3. 单发射乱序处理器核心设计。实现了前后端解耦的单发射乱序处理器，支持 RV64IM 指令集。后端采用统一物理寄存器文件的寄存器重命名方案，通过 FutureRAT/ArchRAT/FreeList/BusyTable 协同工作实现乱序执行与顺序提交；发射队列与执行单元采用泛型抽象类设计，具备良好的代码复用性与可扩展性。前端分支预测采用 GShare+IJTC+RAS 组合方案，在 MicroBench 测试套件下达到 87% 的预测准确率。
4. 缓存子系统设计。实现了 PIPT 结构的 ICache 与 DCache，均为 4 路组相联、64 组、64 字节缓存行，采用 Pseudo-LRU 替换策略。处理器核心侧采用自定义类 SRAM 总线协议降低接口复杂度，缓存对外通过 AXI4 Burst 完成 refill 与 writeback。DCache 支持写回策略、Write Buffer 优化与 fence.i 全缓存刷写。
5. RISC-V 特权级架构与虚拟内存实现。实现了 M/S/U 三级特权模式、完整的 CSR 寄存器集合、异常委托机制、外部/定时器/软件三类中断源的注入与处理，以及 Sv39 三级页表虚拟内存机制（含 TLB 与 PTW），为操作系统运行提供了完整的体系结构支持。
6. 仿真验证平台构建。基于 Verilator 与 C++20 构建了功能完备的仿真验证平台，包含类 GDB 简易调试器（内置 Flex/Bison 表达式求值引擎）、四类执行追踪器（ITrace/MTrace/DTrace/FTrace）、基于 Spike 的逐指令差分测试、基于 fork 的 LightSSS 快照回放机制，以及 NVBoard 外设可视化集成。
7. 操作系统运行验证。处理器已成功运行 xv6 教学操作系统与 RT-Thread 实时操作系统，验证了乱序执行、缓存访问、异常中断处理与虚拟内存管理等关键功能的正确性。

本项目工作的局限性与未来改进方向包括：当前处理器为单发射结构，指令吞吐率受限于每周期至多提交一条指令，后续可扩展为双发射或多发射以提升性能；前端采用状态机而非流水线结构，取指带宽存在提升空间，可引入流水化取指与 BTB 结构；当前缓存层次仅有 L1，可增加 L2 缓存以降低主存访问延迟；乘除法单

元采用多周期迭代实现，可替换为流水化设计以提升吞吐率。此外，可进一步完成FPGA 综合与上板验证，评估实际时钟频率与资源占用。

参考文献

[1] Patterson D A, Waterman A. The RISC-V Reader: An Open Architecture Atlas[M]. 1.0.0th ed. Berkeley, California: Strawberry Canyon LLC, 2017.

[2] 乐鑫科技. 乐鑫 ESP32-C3 功能介绍[Z].

[3] Sharma A. How NVIDIA Shipped One Billion RISC-V Cores In 2024[Z]. 2025.

[4] 电子工程专辑. 国家八部门联合起草指导政策，鼓励全国使用开源 RISC-V 芯片[Z].

[5] 中国软件行业协会. 中国软件产业高质量发展报告（2024）[Z].

[6] Nuclei System Technology. Hummingbirdv2 E203 Core and SoC[Z].

[7] Zhao J, Korpan B, Gonzalez A, et al. SonicBOOM: The 3rd Generation Berkeley Out-of-Order Machine[C]//Fourth Workshop on Computer Architecture Research with RISC-V (CARRV). 2020.

[8] Wang K, Xu Y, Yu Z, et al. XiangShan Open-Source High Performance RISC-V Processor Design and Implementation[J/OL]. Journal of Computer Research and Development, 2023, 60(3): 476-493. DOI:10.7544/issn1000-1239.202221036.

[9] 雷思磊. 自己动手写 CPU[M]. 电子工业出版社, 2014.

[10] 汪文祥. CPU 设计实战[M]. 机械工业出版社, 2021.

[11] 姚永斌. 超标量处理器设计[M]. 清华大学出版社, 2014.

[12] Smith J E, Sohi G S. The Microarchitecture of Superscalar Processors[J/OL]. Proceedings of the IEEE, 1995, 83(12): 1609-1624. DOI:10.1109/5.476078.

[13] Smith J E, Pleszkun A R. Implementing Precise Interrupts in Pipelined Processors[J/OL]. IEEE Transactions on Computers, 1988, 37(5): 562-573. DOI:10.1109/12.4607.

[14] Hwu W m W, Patt Y N. Checkpoint Repair for High-Performance Out-of-Order Execution Machines[J/OL]. IEEE Transactions on Computers, 1987(12): 1496-1514. DOI:10.1109/TC.1987.5009500.

[15] Kessler R E. The Alpha 21264 Microprocessor[J/OL]. IEEE Micro, 1999, 19(2): 24-36. DOI:10.1109/40.755465.

[16] Yeager K C. The MIPS R10000 Superscalar Microprocessor[J/OL]. IEEE Micro, 1996, 16(2): 28-41. DOI:10.1109/40.491460.

[17] McFarling S. Combining Branch Predictors: TN-36[R]. 1993.

[18] Razilov V, Matúš E, Fettweis G. Implementation of the Tagged Geometric History Length Access Interval Predictor[C/OL]//2024 IEEE Nordic Circuits and Systems Conference (NorCAS). 2024: 1-4. DOI:10.1109/NorCAS64408.2024.10752438.

[19] Bachrach J, Vo H, Richards B, et al. Chisel: Constructing Hardware in a Scala Embedded Language[C/OL]//Proceedings of the 49th Annual Design

Automation Conference. San Francisco, California: ACM, 2012: 1216-1225. DOI:10.1145/2228360.2228584.

[20] You K, Xu Y, Feng K, et al. DiffTest-H: Toward Semantic-Aware Communication in Hardware-Accelerated Processor Verification[C/OL]//Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture. ACM, 2025: 1462-1476. DOI:10.1145/3725843.3756108.

[21] RISC-V Software. riscv-isa-sim (Spike)[Z]. 2026.

[22] Cox R, Kaashoek M F, Morris R. xv6: A Simple, Unix-like Teaching Operating System[R]. 2024.

[23] Math S S, B M R. Design of AMBA AXI4 Protocol for System-on-Chip Communication[J/OL]. International Journal of Communication Networks and Security, 2012: 38-42. DOI:10.47893/IJCNS.2012.1033.

[24] Yi Q, Puxiang X, Zhu T. The Design and Implementation of the RT-Thread Operating System[M]. CRC Press, 2023.

[25] Choi M, Park J H, Jeong Y S. Revisiting Reorder Buffer Architecture for Next Generation High Performance Computing[J/OL]. The Journal of Supercomputing, 2013, 65(2): 484-495. DOI:10.1007/s11227-011-0734-x.

[26] Tomasulo R M. An Efficient Algorithm for Exploiting Multiple Arithmetic Units[J/OL]. IBM Journal of Research and Development, 1967, 11(1): 25-33. DOI:10.1147/rd.111.0025.

[27] Waterman A, Asanović K. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture[R]. 2024.

[28] RISC-V International. RISC-V Supervisor Binary Interface Specification[R]. 2024.

[29] RISC-V International. RISC-V Platform-Level Interrupt Controller Specification[R]. 2024.

[30] SiFive. SiFive Interrupt Cookbook[R]. 2024.

[31] ARM. Introduction to AMBA AXI4[R]. 2020.

[32] Asanović K, Avizienis R, Bachrach J, et al. The Rocket Chip Generator: UCB/EECS-2016–17[R]. 2016.

[33] Belady L A. A Study of Replacement Algorithms for a Virtual-Storage Computer[J/OL]. IBM Systems Journal, 1966, 5(2): 78-101. DOI:10.1147/sj.52.0078.

攻读学士学位期间取得创新性成果

参与的科研项目及获奖情况

[1] 2024 年全国大学生计算机系统能力大赛-编译系统实现赛, 三等奖

哈尔滨工业大学本科毕业论文（设计）

原创性声明和使用权限

本科毕业论文（设计）原创性声明

本人郑重声明：此处所提交的本科毕业论文（设计）《RISC-V 指令集乱序处理器设计》，是本人在导师指导下，在哈尔滨工业大学攻读学士学位期间独立进行研究工作所取得的成果，且毕业论文（设计）中除已标注引用文献的部分外不包含他人完成或已发表的研究成果。对本毕业论文（设计）的研究工作做出重要贡献的个人和集体，均已在文中以明确方式注明。本毕业论文（设计）对使用 AI 工具的情况进行了明确标注。

作者签名：王新　　日期：1980 年 1 月 1 日

本科毕业论文（设计）使用权限

本科毕业论文（设计）是本科生在哈尔滨工业大学攻读学士学位期间完成的成果，知识产权归属哈尔滨工业大学。本科毕业论文（设计）的使用权限如下：

（1）学校可以采用影印、缩印或其他复制手段保存本科生上交的毕业论文（设计），并向有关部门报送本科毕业论文（设计）；（2）根据需要，学校可以将本科毕业论文（设计）部分或全部内容编入有关数据库进行检索和提供相应阅览服务；（3）本科生毕业后发表与此毕业论文（设计）研究成果相关的学术论文和其他成果时，应征得导师同意，且第一署名单位为哈尔滨工业大学。

保密论文在保密期内遵守有关保密规定，解密后适用于此使用权限规定。

本人知悉本科毕业论文（设计）的使用权限，并将遵守有关规定。

作者签名：　　日期：　年　月　日
1980　1　1

导师签名：王新　　日期：　年　月　日
Lorem ipsum dolor.　　1980　1　1

致　谢

本科四年的学习与本项目的完成，离不开众多师长和同学的帮助与支持，在此谨致诚挚的感谢。

首先，衷心感谢我的毕业设计指导教师薛睿老师。从数字逻辑设计实验、计算机组成原理实验到计算机设计与实践，再到本次毕业设计，薛老师在处理器设计的各个阶段给予了我系统而耐心的指导，为本项目工作奠定了坚实的基础。

感谢夏文老师和胡皓学长在科研方面对我的悉心指导，帮助我建立了严谨的研究思维与方法论。

感谢郑伟杰老师在学业规划与日常生活中给予的关心与指导，使我在大学期间受益良多。

感谢王鸿鹏老师在计算机体系结构领域的深入指导，拓展了我对处理器设计与系统架构的理解视野。

感谢汪花梅老师、杨嘉辉学长、龙家增学长、饶川同学和梁韬学长在编译器比赛中的指导与协作，这段经历极大地锻炼了我的工程实践能力与团队合作意识。

最后，感谢家人在求学路上始终如一的理解与支持。